Redes Neurais Convolucionais na Classificação de Moda

Relatório Técnico: Avaliação dos 13 Modelos

Anna Victória Duarte Barbosa

Orientador: Fabrízzio Condé de Oliveira

Trabalho de Conclusão de Curso

Curso de Bacharelado em Matemática
ênfase em Matemática Aplicada e Computacional

Tarefa	Classificação binária: Bolsas *vs.* Calças
Protocolo	Repeated Holdout (10 sementes)
Modelos	13 (2 CNN do zero + 11 Transfer Learning)
Framework	TensorFlow / Keras
Plataforma	Google Colab (GPU)

14 de abril de 2026

0 Sumário

1 Introdução **5**

2 Fundamentos Matemáticos **5**
2.1 Operação de Convolução 2D 5
2.2 Funções de Ativação 6
2.2.1 ReLU — *Rectified Linear Unit* 6
2.2.2 Logit — Saída Linear Bruta da Rede 6
2.2.3 Softmax — Conversão para Probabilidade 6
2.3 Operações de Pooling 7
2.3.1 Max Pooling 7
2.3.2 Global Average Pooling (GAP) 7
2.4 Função de Perda: Entropia Cruzada Categórica 7
2.5 Exemplo Numérico: Softmax, Probabilidade e Função de Perda 8
2.6 Otimizador Adam 9
2.7 Convolução Separável em Profundidade 10
2.8 Normalização em Lote (*Batch Normalization*) 10
2.9 Estatísticas de Replicabilidade 11

3 Metodologia Experimental **11**
3.1 Protocolo de Avaliação: Repeated Holdout 11
3.2 Conjunto de Dados e Divisão 11
3.3 Aumento de Dados (*Data Augmentation*) 12
3.4 Pré-processamento de Imagens 12
3.5 Callback: Early Stopping 12
3.6 Métricas de Avaliação 13
3.7 Intervalo de Confiança 13
3.8 Configuração Geral dos 13 Modelos 13
3.9 Arquitetura da Cabeça de Classificação (Transfer Learning) 13

4 Modelo 1 — CNN Treinada do Zero (lr $= 5 \times 10^{-4}$) **14**
4.1 Descrição da Arquitetura 14
4.2 Hiperparâmetros 14
4.3 Resultados por Rodada 15
4.4 Estatísticas Resumo 15
4.5 Gráficos 15
4.6 Análise 16

5 Modelo 2 — CNN Treinada do Zero (lr $= 1 \times 10^{-4}$) **17**
5.1 Descrição da Arquitetura 17
5.2 Hiperparâmetros 17
5.3 Resultados por Rodada 17
5.4 Estatísticas Resumo 17
5.5 Gráficos 18
5.6 Análise 19

6 Modelo 3 — MobileNetV2 **19**
6.1 Descrição do Backbone . . . 19
6.2 Hiperparâmetros . . . 20
6.3 Resultados por Rodada . . . 20
6.4 Estatísticas Resumo . . . 20
6.5 Gráficos . . . 21
6.6 Análise . . . 22

7 Modelo 4 — EfficientNetB0 **22**
7.1 Descrição do Backbone . . . 22
7.2 Hiperparâmetros . . . 22
7.3 Resultados por Rodada . . . 23
7.4 Estatísticas Resumo . . . 23
7.5 Gráficos . . . 23
7.6 Análise . . . 24

8 Modelo 5 — EfficientNetB1 **25**
8.1 Descrição do Backbone . . . 25
8.2 Hiperparâmetros . . . 25
8.3 Resultados por Rodada . . . 25
8.4 Estatísticas Resumo . . . 25
8.5 Gráficos . . . 26
8.6 Análise . . . 27

9 Modelo 6 — EfficientNetB2 **27**
9.1 Descrição do Backbone . . . 27
9.2 Hiperparâmetros . . . 27
9.3 Resultados por Rodada . . . 28
9.4 Estatísticas Resumo . . . 28
9.5 Gráficos . . . 28
9.6 Análise . . . 29

10 Modelo 7 — ResNet50 **30**
10.1 Descrição do Backbone . . . 30
10.2 Hiperparâmetros . . . 30
10.3 Resultados por Rodada . . . 30
10.4 Estatísticas Resumo . . . 30
10.5 Gráficos . . . 31
10.6 Análise . . . 32

11 Modelo 8 — DenseNet121 **32**
11.1 Descrição do Backbone . . . 32
11.2 Hiperparâmetros . . . 32
11.3 Resultados por Rodada . . . 33
11.4 Estatísticas Resumo . . . 33
11.5 Gráficos . . . 33
11.6 Análise . . . 34

12 Modelo 9 — MobileNetV3Small **35**
12.1 Descrição do Backbone . . . 35
12.2 Hiperparâmetros . . . 35
12.3 Resultados por Rodada . . . 35
12.4 Estatísticas Resumo . . . 35
12.5 Gráficos . . . 36
12.6 Análise . . . 37

13 Modelo 10 — MobileNetV3Large **37**
13.1 Descrição do Backbone . . . 37
13.2 Hiperparâmetros . . . 37
13.3 Resultados por Rodada . . . 38
13.4 Estatísticas Resumo . . . 38
13.5 Gráficos . . . 38
13.6 Análise . . . 39

14 Modelo 11 — NASNetMobile **40**
14.1 Descrição do Backbone . . . 40
14.2 Hiperparâmetros . . . 40
14.3 Resultados por Rodada . . . 40
14.4 Estatísticas Resumo . . . 40
14.5 Gráficos . . . 41
14.6 Análise . . . 42

15 Modelo 12 — InceptionV3 **42**
15.1 Descrição do Backbone . . . 42
15.2 Hiperparâmetros . . . 42
15.3 Resultados por Rodada . . . 43
15.4 Estatísticas Resumo . . . 43
15.5 Gráficos . . . 43
15.6 Análise . . . 44

16 Modelo 13 — Xception **45**
16.1 Descrição do Backbone . . . 45
16.2 Hiperparâmetros . . . 45
16.3 Resultados por Rodada . . . 45
16.4 Estatísticas Resumo . . . 45
16.5 Gráficos . . . 46
16.6 Análise . . . 47

17 Análise Comparativa dos 13 Modelos **48**
17.1 Tabela Comparativa de Médias — Métricas de Classificação . . . 48
17.2 Tabela Comparativa de Médias — Tempos de Execução . . . 48
17.3 Tabela Resumo Consolidada dos 13 Modelos . . . 48
17.4 Discussão Geral . . . 50
17.4.1 Modelos de Melhor Desempenho . . . 50
17.4.2 Família EfficientNet . . . 50

17.4.3 Modelos Leves (*Mobile*) 50
17.4.4 Impacto da Arquitetura CNN do Zero 50
17.4.5 Relação Desempenho × Tempo 50

18 Conclusões **51**

Referências **51**

1 Introdução

Este relatório documenta a avaliação experimental de **13 modelos de Redes Neurais Convolucionais (CNNs)** aplicados à classificação binária de imagens de moda, distinguindo **bolsas** de **calças**. Os modelos compreendem duas categorias:

- **CNNs treinadas do zero** (*from scratch*): redes convolucionais projetadas e treinadas exclusivamente com os dados da tarefa, sem aproveitamento de pesos pré-treinados. Dois valores de taxa de aprendizado foram avaliados: 5×10^{-4} e 1×10^{-4}.
- **Transfer Learning (TL)**: 11 modelos com *backbones* pré-treinados na base ImageNet, cujas camadas convolucionais são congeladas e apenas a cabeça de classificação é treinada nos dados da tarefa.

O objetivo é identificar qual arquitetura oferece o melhor compromisso entre desempenho preditivo (acurácia, precisão, recall e F1-score) e custo computacional (tempo de treinamento e de inferência).

2 Fundamentos Matemáticos

Esta seção apresenta as principais funções e expressões matemáticas que fundamentam o funcionamento das CNNs avaliadas neste trabalho, desde a operação primitiva de convolução até o otimizador e as funções estatísticas de replicabilidade. As referências utilizadas incluem obras seminais de aprendizado profundo [1, 3, 2] e artigos específicos de cada componente, detalhados em cada subseção.

2.1 Operação de Convolução 2D

A operação central de uma CNN é a **convolução discreta 2D**, que desliza um filtro (kernel) K de dimensão $k_h \times k_w$ sobre uma imagem de entrada I e produz um mapa de características:

$$(I * K)[i, j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I[i+m,\ j+n] \cdot K[m, n] \tag{1}$$

onde i e j indexam a posição espacial no mapa de saída. Em redes com múltiplos canais de entrada C e N filtros paralelos, a equação se generaliza para:

$$Z^{(n)}[i, j] = \sum_{c=1}^{C} \sum_{m=0}^{k_h-1} \sum_{n'=0}^{k_w-1} X^{(c)}[i+m,\ j+n'] \cdot K^{(n,c)}[m, n'] + b^{(n)} \tag{2}$$

onde $b^{(n)}$ é o viés do n-ésimo filtro. Neste projeto, todos os filtros convolucionais utilizam núcleos 3×3. A operação de convolução aplicada ao reconhecimento de padrões visuais foi formalizada por LeCun et al. [1], consolidando-se como o bloco fundamental das CNNs modernas [3].

2.2 Funções de Ativação

2.2.1 ReLU — *Rectified Linear Unit*

Após cada camada convolucional, aplica-se a função de ativação **ReLU**, que introduz não-linearidade ao modelo zerando todos os valores negativos:

$$\mathrm{ReLU}(x) = \max(0,\, x) \tag{3}$$

A ReLU é computacionalmente eficiente e mitiga o problema do gradiente desvanecente em redes profundas [5], sendo a ativação padrão em todas as camadas ocultas dos 13 modelos. Nair e Hinton [4] demonstraram que unidades com ativação retificada superam funções sigmoidais em tarefas de classificação, e Glorot et al. [5] evidenciaram sua capacidade de induzir representações esparsas, favorecendo a generalização.

2.2.2 Logit — Saída Linear Bruta da Rede

Antes de aplicar qualquer função de ativação, a última camada densa da rede (`Dense(2)`) computa uma combinação linear dos valores recebidos:

$$z_c = \mathbf{w}_c^\top \mathbf{h} + b_c, \qquad c = 1, \ldots, C \tag{4}$$

onde $\mathbf{h}$ é o vetor de ativações da camada anterior (saída do `Dense(64)`), $\mathbf{w}_c$ é o vetor de pesos associado à classe c e b_c é o viés correspondente.

O valor z_c é chamado de **logit** da classe c. O termo vem da função *logit* da estatística ($\log \frac{p}{1-p}$), originalmente proposta no contexto de modelos de regressão para dados ordinais [6], mas no contexto de redes neurais designa genericamente qualquer **pontuação linear irrestrita** produzida antes de uma função de ativação — podendo assumir qualquer valor real ($z_c \in \mathbb{R}$), positivo ou negativo, sem interpretação direta de probabilidade [3].

O vetor de logits $\mathbf{z} = (z_1, z_2)$ para a tarefa binária deste trabalho representa, portanto, o *grau de afinidade* bruto que a rede atribui a cada classe. Um logit maior indica maior tendência da rede em classificar a entrada naquela classe, mas apenas após a Softmax esses valores se tornam probabilidades comparáveis e somam 1.

2.2.3 Softmax — Conversão para Probabilidade

A camada de saída de **todos os 13 modelos** utiliza a função **Softmax**, que converte o vetor de logits $\mathbf{z} = (z_1, z_2, \ldots, z_C)$ em uma distribuição de probabilidade sobre as C classes:

$$\hat{p}_c = \mathrm{softmax}(z_c) = \frac{e^{z_c}}{\displaystyle\sum_{j=1}^{C} e^{z_j}}, \qquad c = 1, \ldots, C \tag{5}$$

A Softmax garante que $\hat{p}_c \in (0, 1)$ para todo c e que $\sum_{c=1}^{C} \hat{p}_c = 1$. Para a tarefa binária deste trabalho ($C = 2$: bolsas e calças), as duas probabilidades complementares são:

$$\hat{p}_{\text{bolsa}} = \frac{e^{z_1}}{e^{z_1} + e^{z_2}}, \qquad \hat{p}_{\text{calça}} = \frac{e^{z_2}}{e^{z_1} + e^{z_2}} \tag{6}$$

A classe predita é determinada por $\hat{y} = \arg\max_c \hat{p}_c$. A interpretação probabilística da saída Softmax em redes de classificação foi formalizada por Bridle [7], tornando-se a função de ativação de saída padrão para classificação multiclasse [3].

2.3 Operações de Pooling

2.3.1 Max Pooling

O **Max Pooling** reduz a dimensão espacial do mapa de características retendo o valor máximo em cada janela $p \times p$ (neste projeto $p = 2$) com passo (*stride*) igual a p:

$$y[i, j] = \max_{\substack{0 \le m < p \\ 0 \le n < p}} x[p\,i + m,\ p\,j + n] \tag{7}$$

Cada bloco Conv+MaxPool na CNN do zero reduz pela metade a resolução espacial ($H \times W \to \frac{H}{2} \times \frac{W}{2}$), expandindo progressivamente o campo receptivo da rede [1]. Zeiler e Fergus [8] demonstraram por meio de visualizações que as camadas de pooling são essenciais para construir invariância a pequenas translações e deformações da entrada.

2.3.2 Global Average Pooling (GAP)

Nos modelos de Transfer Learning, o backbone produz um tensor $F \in \mathbb{R}^{H \times W \times C}$. A camada `GlobalAveragePooling2D` colapsa as dimensões espaciais calculando a média por canal, gerando um vetor $\mathbf{g} \in \mathbb{R}^C$:

$$g_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} F[i, j, c], \qquad c = 1, \ldots, C \tag{8}$$

O GAP foi introduzido por Lin et al. [9] como alternativa às camadas totalmente conectadas, eliminando a necessidade de camadas densas volumosas sobre o mapa de características e atuando como regularizador natural que reduz o risco de *overfitting*.

2.4 Função de Perda: Entropia Cruzada Categórica

Todos os modelos são treinados minimizando a **Entropia Cruzada Categórica** (*Categorical Cross-Entropy*), cuja fundamentação teórica remonta à teoria da informação de Shannon [10] e que mede a divergência entre a distribuição real $\mathbf{y}$ (codificada em *one-hot*) e a distribuição prevista $\hat{\mathbf{p}}$ (saída da Softmax) [3]:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{i,c} \log(\hat{p}_{i,c}) \tag{9}$$

onde N é o número de amostras no mini-batch, $y_{i,c} \in \{0, 1\}$ indica se a amostra i pertence à classe c, e $\hat{p}_{i,c}$ é a probabilidade prevista pela Softmax. Como $\mathbf{y}$ é *one-hot*, apenas o termo da classe verdadeira c_i^* contribui para a soma:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log\left(\hat{p}_{i,\,c_i^*}\right) \tag{10}$$

A perda é minimizada quando $\hat{p}_{i,c_i^*} \to 1$, isto é, quando o modelo atribui probabilidade máxima à classe correta. O Early Stopping monitora $\mathcal{L}$ na faixa de treino com `patience` = 5.

2.5 Exemplo Numérico: Softmax, Probabilidade e Função de Perda

Para concretizar a relação entre logits, Softmax e Entropia Cruzada, considere uma imagem de **bolsa** (classe verdadeira $c^* =$ bolsa, codificação *one-hot* $\mathbf{y} = [1,\ 0]$) passando pela camada de saída `Dense(2, activation='softmax')`, que produz dois logits $\mathbf{z} = (z_{\text{bolsa}},\ z_{\text{calça}})$. Analisamos três cenários que ilustram como a Softmax converte logits em probabilidades e como a Entropia Cruzada penaliza cada situação.

Cenário 1 — Predição correta (modelo confiante):

$$\mathbf{z} = (2{,}10;\ 0{,}30)$$

$$\hat{p}_{\text{bolsa}} = \frac{e^{2{,}10}}{e^{2{,}10} + e^{0{,}30}} = \frac{8{,}166}{8{,}166 + 1{,}350} = \frac{8{,}166}{9{,}516} \approx 0{,}858$$

$$\hat{p}_{\text{calça}} = \frac{e^{0{,}30}}{9{,}516} = \frac{1{,}350}{9{,}516} \approx 0{,}142$$

$$\mathcal{L} = -\log(\hat{p}_{c^*}) = -\log(0{,}858) \approx \mathbf{0{,}153}$$

A perda é baixa: o modelo atribui 85,8 % de probabilidade à classe correta. O gradiente resultante é pequeno e o Adam realiza uma atualização sutil dos pesos.

Cenário 2 — Predição incerta (modelo sem confiança):

$$\mathbf{z} = (0{,}50;\ 0{,}50)$$

$$\hat{p}_{\text{bolsa}} = \frac{e^{0{,}50}}{e^{0{,}50} + e^{0{,}50}} = \frac{1{,}649}{3{,}297} = 0{,}500$$

$$\hat{p}_{\text{calça}} = \frac{1{,}649}{3{,}297} = 0{,}500$$

$$\mathcal{L} = -\log(0{,}500) \approx \mathbf{0{,}693}$$

Logits iguais produzem distribuição uniforme — máxima incerteza. A perda moderada indica que o modelo ainda não aprendeu a distinguir as classes, situação típica das primeiras épocas de treinamento.

Cenário 3 — Predição errada (modelo confiante na classe errada):

$$\mathbf{z} = (-1{,}20;\ 2{,}50)$$

$$\hat{p}_{\text{bolsa}} = \frac{e^{-1{,}20}}{e^{-1{,}20} + e^{2{,}50}} = \frac{0{,}301}{0{,}301 + 12{,}182} = \frac{0{,}301}{12{,}483} \approx 0{,}024$$

$$\hat{p}_{\text{calça}} = \frac{12{,}182}{12{,}483} \approx 0{,}976$$

$$\mathcal{L} = -\log(0{,}024) \approx \mathbf{3{,}730}$$

A perda é alta: apenas 2,4 % de probabilidade foi atribuída à classe correta. O gradiente é grande, forçando o Adam a realizar uma correção expressiva nos pesos θ da rede.

A Tabela 1 resume os três cenários e evidencia a relação monotônica entre *confiança na classe errada* e *magnitude da perda*:

Tabela 1: Resumo do exemplo numérico — logits, probabilidades Softmax e Entropia Cruzada para uma imagem de bolsa ($\mathbf{y} = [1, 0]$)

Cenário	z_{bolsa}	$z_{\text{calça}}$	$\hat{p}_{\text{bolsa}}$	$\hat{p}_{\text{calça}}$	$\mathcal{L}$	**Situação**
Predição correta	2,10	0,30	0,858	0,142	0,153	✓
Predição incerta	0,50	0,50	0,500	0,500	0,693	∼
Predição errada	−1,20	2,50	0,024	0,976	3,730	×

Interpretação geral: à medida que o treinamento progride, o Adam minimiza $\mathcal{L}$ ajustando os pesos para que o logit da classe correta cresça e o da classe errada diminua, empurrando $\hat{p}_{c^*} \to 1$ e $\mathcal{L} \to 0$. Para o modelo *perfeito* ($\hat{p}_{c^*} = 1$), a perda seria $-\log(1) = 0$; para o modelo totalmente errado ($\hat{p}_{c^*} \to 0$), a perda tenderia a $+\infty$.

2.6 Otimizador Adam

O treinamento utiliza o otimizador **Adam** (*Adaptive Moment Estimation*) [15], que adapta individualmente a taxa de aprendizado de cada parâmetro com base nos primeiros e segundos momentos dos gradientes.

Seja $g_t = \nabla_\theta \mathcal{L}_t$ o gradiente no passo t. As equações de atualização são:

$$m_t = \beta_1\, m_{t-1} + (1 - \beta_1)\, g_t \tag{11}$$

$$v_t = \beta_2\, v_{t-1} + (1 - \beta_2)\, g_t^2 \tag{12}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \tag{13}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{14}$$

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{15}$$

onde m_t é o **primeiro momento** (média exponencial dos gradientes), v_t é o **segundo momento** (variância exponencial), e as correções $\hat{m}_t$, $\hat{v}_t$ compensam o viés de inicialização em zero. Os hiperparâmetros padrão do Keras são $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon = 10^{-7}$, e α é a taxa de aprendizado (10^{-4} para os modelos de TL; 5×10^{-4} ou 10^{-4} para as CNNs do zero).

2.7 Convolução Separável em Profundidade

Os modelos da família MobileNet (V2, V3Small e V3Large) e o Xception substituem as convoluções padrão por **convoluções separáveis em profundidade** (*Depthwise Separable Convolutions*), introduzidas por Howard et al. [12] e popularizadas no Xception por Chollet [23], compostas de dois passos sequenciais:

(1) Depthwise Convolution — aplica um filtro independente por canal de entrada:

$$\hat{Z}^{(c)}[i,j] = \sum_m \sum_n X^{(c)}[i+m,\, j+n] \cdot K_{\text{dw}}^{(c)}[m,n] \tag{16}$$

(2) Pointwise Convolution — combina os canais com uma convolução 1×1:

$$Z^{(n)}[i,j] = \sum_{c=1}^{C} \hat{Z}^{(c)}[i,j] \cdot K_{\text{pw}}^{(n,c)} \tag{17}$$

Sendo D_K o tamanho do kernel, C os canais de entrada, N o número de filtros e $D_F \times D_F$ a resolução do mapa, a razão de custo computacional entre as duas abordagens é:

$$\frac{\mathcal{C}_{\text{sep}}}{\mathcal{C}_{\text{padro}}} = \frac{D_K^2 C D_F^2 + C N D_F^2}{D_K^2 C N D_F^2} = \frac{1}{N} + \frac{1}{D_K^2} \tag{18}$$

Para $D_K = 3$ e $N \gg 1$, a redução é de aproximadamente **8**×, viabilizando o uso em dispositivos com restrições de memória e processamento.

2.8 Normalização em Lote (*Batch Normalization*)

Vários backbones (ResNet50, EfficientNet, DenseNet121 e outros) aplicam **Batch Normalization** [11] após as convoluções, normalizando as ativações dentro do mini-batch para reduzir o *internal covariate shift* — fenômeno descrito por Ioffe e Szegedy [11] como a mudança na distribuição das ativações ao longo do treinamento, que desacelera a convergência:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \tag{19}$$

onde μ_B e σ_B^2 são a média e a variância das ativações no batch atual. Em seguida, uma transformação afim com parâmetros aprendíveis γ (escala) e β (deslocamento) restaura a capacidade expressiva:

$$y_i = \gamma\, \hat{x}_i + \beta \tag{20}$$

2.9 Estatísticas de Replicabilidade

O protocolo Repeated Holdout com $n = 10$ rodadas exige a quantificação da variabilidade dos resultados [13, 14]. As seguintes medidas são reportadas para cada métrica:

Desvio Padrão Amostral:

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{21}$$

Coeficiente de Variação (CV):

$$\mathrm{CV} = \frac{s}{\bar{x}} \times 100\% \tag{22}$$

O CV expressa a dispersão relativa à média, permitindo comparar a estabilidade entre modelos com desempenhos médios distintos. Valores baixos de CV indicam maior estabilidade frente à variação das sementes.

Intervalo de Confiança de 95% (t-Student):

$$\mathrm{IC}_{95\%} = \bar{x} \pm t_{0{,}025,\, n-1} \cdot \frac{s}{\sqrt{n}} \tag{23}$$

Para $n = 10$ rodadas utiliza-se $t_{0{,}025,\, 9} = 2{,}262$. Dietterich [13] recomenda o uso de testes estatísticos baseados em reamostragem para comparação de algoritmos de aprendizado, sendo o IC baseado em t-Student uma das abordagens mais adotadas na literatura de aprendizado de máquina [14].

3 Metodologia Experimental

3.1 Protocolo de Avaliação: Repeated Holdout

O protocolo **Repeated Holdout** com 10 sementes aleatórias fixas (SEED $\in \{0, 1, \ldots, 9\}$) foi adotado para garantir a **replicabilidade** e estimar a variabilidade dos resultados. Em cada rodada, uma semente diferente é utilizada para inicializar os geradores aleatórios do Python, NumPy e TensorFlow, assegurando que os resultados sejam reprodutíveis.

3.2 Conjunto de Dados e Divisão

- **Treinamento**: imagens nas pastas `BOLSAS TREINAMENTO` e `CALÇAS TREINAMENTO`, localizadas em `TREINAMENTO/`.
- **Teste**: imagens nas pastas `BOLSAS TESTE` e `CALÇAS TESTE`, localizadas em `TESTE/`.
- A divisão treino/teste é *fixa*; apenas a semente de embaralhamento do gerador de treino varia entre rodadas.

3.3 Aumento de Dados (*Data Augmentation*)

Dois conjuntos de parâmetros de aumento foram utilizados:

Tabela 2: Parâmetros de aumento de dados por tipo de modelo

Parâmetro	CNN do Zero	Transfer Learning
`rotation_range`	15	15
`zoom_range`	0.25	0.20
`width_shift_range`	0.10	0.10
`height_shift_range`	0.10	0.10
`shear_range`	0.20	—
`horizontal_flip`	✓	✓
`brightness_range`	[0.8, 1.2]	—

3.4 Pré-processamento de Imagens

Cada família de backbone exige uma função de pré-processamento específica, que normaliza os pixels para a escala esperada durante o pré-treino no ImageNet. As CNNs do zero utilizam rescaling simples ($\div 255$).

Tabela 3: Funções de pré-processamento por backbone

Backbone	Função de pré-processamento
CNN do zero	`rescale = 1/255`
MobileNetV2	`mobilenet_v2.preprocess_input`
EfficientNetB0/B1/B2	`efficientnet.preprocess_input`
ResNet50	`resnet.preprocess_input`
DenseNet121	`densenet.preprocess_input`
MobileNetV3Small/Large	`mobilenet_v3.preprocess_input`
NASNetMobile	`nasnet.preprocess_input`
InceptionV3	`inception_v3.preprocess_input`
Xception	`xception.preprocess_input`

3.5 Callback: Early Stopping

Todos os modelos utilizam `EarlyStopping` com os seguintes parâmetros:

- `monitor = 'loss'` (monitora a perda de treino)
- `patience = 5` (aguarda 5 épocas sem melhoria antes de parar)
- `restore_best_weights = True`

3.6 Métricas de Avaliação

Quatro métricas de classificação são calculadas com média ponderada (*weighted average*):

$$\text{Acurácia} = \frac{TP + TN}{TP + TN + FP + FN} \tag{24}$$

$$\text{Precisão} = \frac{TP}{TP + FP} \tag{25}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{26}$$

$$\text{F1-score} = \frac{2 \cdot \text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}} \tag{27}$$

Além disso, são registrados os tempos de treinamento, inferência e total (em segundos).

3.7 Intervalo de Confiança

O intervalo de confiança de 95% é calculado usando a distribuição t-Student para amostras pequenas ($n = 10$), com $t_{0.025,9} = 2{,}262$:

$$\text{IC}_{95\%} = \bar{x} \pm t_{0.025,\,n-1} \cdot \frac{s}{\sqrt{n}}$$

3.8 Configuração Geral dos 13 Modelos

Tabela 4: Resumo de configuração de todos os modelos avaliados

ID	Modelo	Tipo	Backbone	Img	Batch	Ép.	LR	Preprocess
1	cnn_from_scratch	CNN	ConvNet	150	32	50	5×10^{-4}	Rescale
2	cnn_from_scratch_lr1e4	CNN	ConvNet	150	32	50	1×10^{-4}	Rescale
3	mobilenetv2	TL	MobileNetV2	150	32	30	1×10^{-4}	MNv2
4	efficientnetb0	TL	EfficientNetB0	224	32	30	1×10^{-4}	EffNet
5	efficientnetb1	TL	EfficientNetB1	240	32	30	1×10^{-4}	EffNet
6	efficientnetb2	TL	EfficientNetB2	260	32	30	1×10^{-4}	EffNet
7	resnet50	TL	ResNet50	224	32	30	1×10^{-4}	ResNet
8	densenet121	TL	DenseNet121	224	32	30	1×10^{-4}	DenseNet
9	mobilenetv3small	TL	MobileNetV3S	224	32	30	1×10^{-4}	MNv3
10	mobilenetv3large	TL	MobileNetV3L	224	32	30	1×10^{-4}	MNv3
11	nasnetmobile	TL	NASNetMobile	224	32	30	1×10^{-4}	NASNet
12	inceptionv3	TL	InceptionV3	299	16	30	1×10^{-4}	Inception
13	xception	TL	Xception	299	16	30	1×10^{-4}	Xception

3.9 Arquitetura da Cabeça de Classificação (Transfer Learning)

Para todos os modelos de Transfer Learning, a mesma cabeça de classificação é adicionada sobre o backbone congelado:

1. **Backbone pré-treinado** (pesos ImageNet, `trainable = False`, `include_top = False`)
2. `GlobalAveragePooling2D()` — reduz o mapa de características a um vetor
3. `Dense(128, activation='relu')`
4. `Dense(64, activation='relu')`
5. `Dense(2, activation='softmax')` — saída binária

Compilação: `Adam(lr=1e-4)`, `loss='categorical_crossentropy'`, `metrics=['accuracy']`.

4 Modelo 1 — CNN Treinada do Zero (lr $= 5 \times 10^{-4}$)

4.1 Descrição da Arquitetura

A CNN do zero (`cnn_from_scratch`) é uma rede convolucional simples, projetada sem aproveitar pesos pré-treinados. Sua arquitetura segue o padrão clássico de empilhamento de blocos Conv+Pool, finalizando com camadas totalmente conectadas:

1. `Conv2D(32, (3,3), activation='relu')` + `MaxPool2D()`
2. `Conv2D(64, (3,3), activation='relu')` + `MaxPool2D()`
3. `Conv2D(128, (3,3), activation='relu')` + `MaxPool2D()`
4. `Flatten()`
5. `Dense(128, activation='relu')`
6. `Dense(64, activation='relu')`
7. `Dense(2, activation='softmax')`

4.2 Hiperparâmetros

Parâmetro	**Valor**
Taxa de Aprendizado	5×10^{-4}
Imagem de Entrada	$150 \times 150 \times 3$
Tamanho do Batch	32
Épocas Máximas	50
Otimizador	Adam
Função de Perda	Categorical Crossentropy
Pré-processamento	Rescale 1/255

4.3 Resultados por Rodada

Tabela 5: Resultados por rodada — CNN do zero (lr = 5×10^{-4})

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
cnn_from_scratch	1	0	0.8167	0.8480	0.8167	0.8124	71.48	33.21	104.69
cnn_from_scratch	2	1	0.6333	0.7400	0.6333	0.5875	41.89	3.28	45.16
cnn_from_scratch	3	2	0.7500	0.8078	0.7500	0.7377	32.43	3.37	35.80
cnn_from_scratch	4	3	0.9500	0.9545	0.9500	0.9499	26.66	4.12	30.78
cnn_from_scratch	5	4	0.6667	0.6667	0.6667	0.6667	21.94	3.72	25.65
cnn_from_scratch	6	5	0.7167	0.7889	0.7167	0.6978	19.96	3.28	23.25
cnn_from_scratch	7	6	0.6833	0.6939	0.6833	0.6790	28.00	4.09	32.10
cnn_from_scratch	8	7	0.7667	0.8409	0.7667	0.7532	26.38	3.38	29.76
cnn_from_scratch	9	8	0.7000	0.8125	0.7000	0.6703	32.36	3.41	35.77
cnn_from_scratch	10	9	0.7167	0.7291	0.7167	0.7128	42.47	3.65	46.12

4.4 Estatísticas Resumo

Tabela 6: Resumo estatístico — CNN do zero (lr = 5×10^{-4})

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.7400	0.0903	0.1221	0.6754	0.8046
Precision	0.7882	0.0846	0.1074	0.7277	0.8488
Recall	0.7400	0.0903	0.1221	0.6754	0.8046
F1_score	0.7267	0.0986	0.1356	0.6562	0.7972
Tempo_Treino_s	34.3572	15.0289	0.4374	23.6069	45.1075
Tempo_Pred_s	6.5507	9.3739	1.4310	-0.1545	13.2559
Tempo_Total_s	40.9079	23.6063	0.5771	24.0222	57.7937

4.5 Gráficos

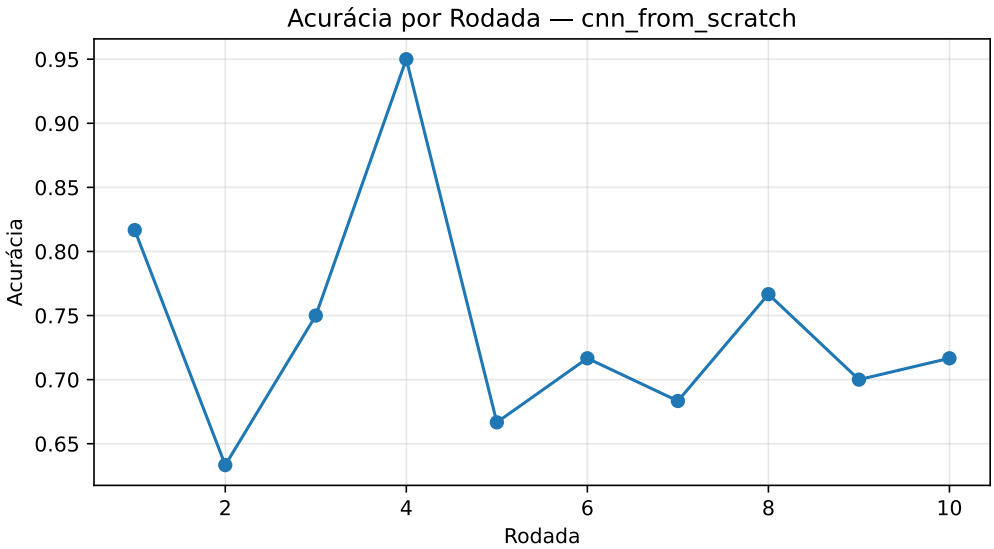

Figura 1: Acurácia por rodada — CNN do zero (lr = 5×10^{-4})

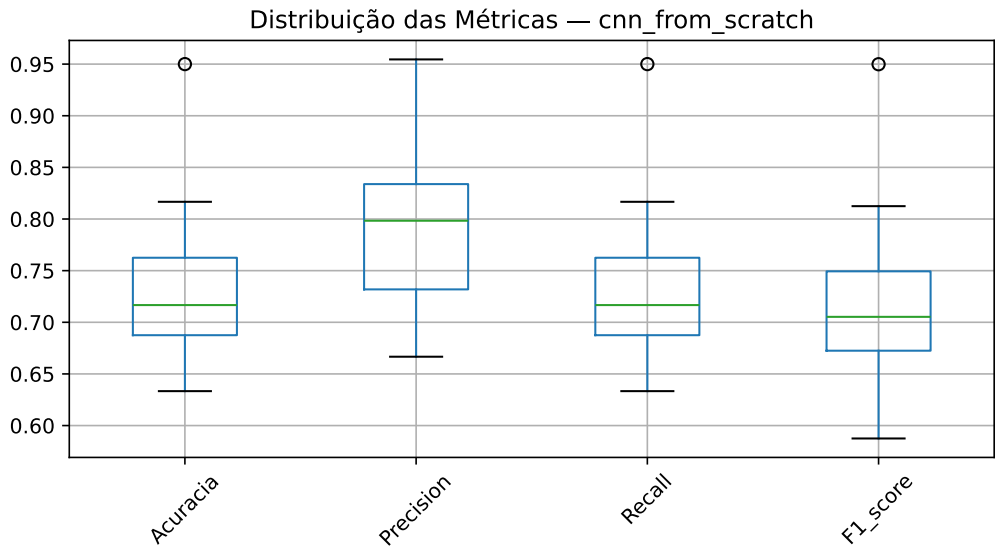

Figura 2: Boxplot das métricas — CNN do zero (lr $= 5 \times 10^{-4}$)

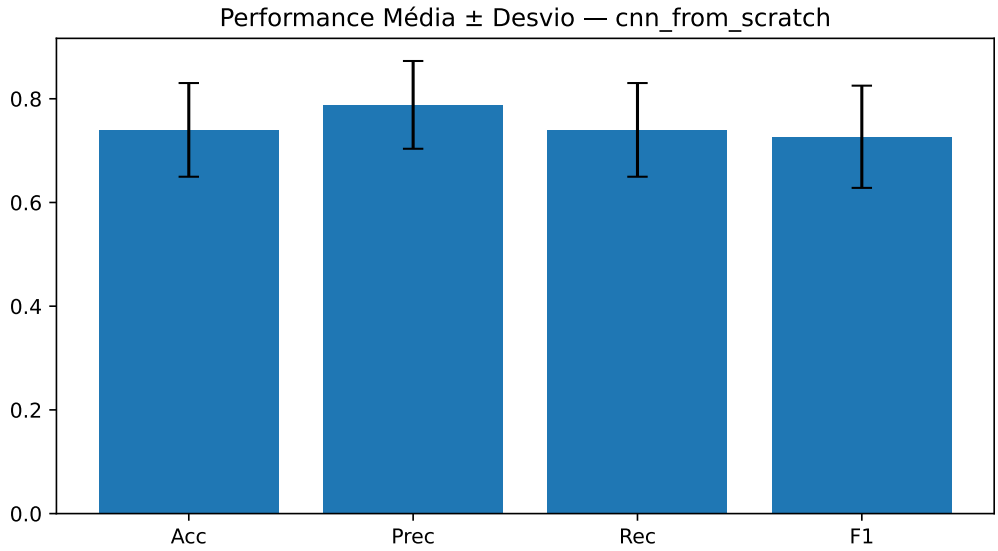

Figura 3: Média ± desvio padrão — CNN do zero (lr $= 5 \times 10^{-4}$)

4.6 Análise

A CNN treinada do zero com lr $= 5 \times 10^{-4}$ apresentou **alta variabilidade** entre as rodadas, com acurácia média de 0,7400 e desvio padrão de 0,0903 (CV = 12,2%). O intervalo de confiança de 95% para a acurácia é [0,6754; 0,8046]. Esse comportamento indica que o modelo é sensível à inicialização dos pesos e à seleção dos dados de treinamento, resultado esperado para redes sem pré-treinamento. A taxa de aprendizado mais alta (5×10^{-4}) pode causar oscilações na otimização.

5 Modelo 2 — CNN Treinada do Zero (lr $= 1 \times 10^{-4}$)

5.1 Descrição da Arquitetura

Arquitetura idêntica ao Modelo 1 (seção 4), diferindo apenas na taxa de aprendizado, que é reduzida para 1×10^{-4}. O objetivo é verificar se um gradiente mais conservador melhora a estabilidade do treinamento.

5.2 Hiperparâmetros

Parâmetro	**Valor**
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$150 \times 150 \times 3$
Tamanho do Batch	32
Épocas Máximas	50
Otimizador	Adam
Função de Perda	Categorical Crossentropy
Pré-processamento	Rescale 1/255

5.3 Resultados por Rodada

Tabela 7: Resultados por rodada — CNN do zero (lr $= 1 \times 10^{-4}$)

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
cnn_from_scratch_lr1e4	1	0	0.8000	0.8571	0.8000	0.7917	74.90	19.48	94.38
cnn_from_scratch_lr1e4	2	1	0.5333	0.7586	0.5333	0.4034	29.17	3.17	32.34
cnn_from_scratch_lr1e4	3	2	0.7000	0.7153	0.7000	0.6946	31.43	3.14	34.57
cnn_from_scratch_lr1e4	4	3	0.7333	0.7512	0.7333	0.7285	46.45	3.25	49.70
cnn_from_scratch_lr1e4	5	4	0.6667	0.6674	0.6667	0.6663	22.49	3.41	25.90
cnn_from_scratch_lr1e4	6	5	0.5833	0.6391	0.5833	0.5369	27.63	3.23	30.86
cnn_from_scratch_lr1e4	7	6	0.6333	0.7083	0.6333	0.5971	45.19	3.22	48.42
cnn_from_scratch_lr1e4	8	7	0.5500	0.7632	0.5500	0.4357	26.59	3.59	30.18
cnn_from_scratch_lr1e4	9	8	0.7167	0.7668	0.7167	0.7027	30.26	3.24	33.50
cnn_from_scratch_lr1e4	10	9	0.6500	0.7941	0.6500	0.6011	35.28	3.14	38.42

5.4 Estatísticas Resumo

Tabela 8: Resumo estatístico — CNN do zero (lr $= 1 \times 10^{-4}$)

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.6567	0.0847	0.1290	0.5961	0.7172
Precision	0.7421	0.0627	0.0845	0.6973	0.7870
Recall	0.6567	0.0847	0.1290	0.5961	0.7172
F1_score	0.6158	0.1266	0.2056	0.5252	0.7064
Tempo_Treino_s	36.9403	15.4200	0.4174	25.9103	47.9704
Tempo_Pred_s	4.8871	5.1295	1.0496	1.2180	8.5563
Tempo_Total_s	41.8275	20.0012	0.4782	27.5204	56.1345

5.5 Gráficos

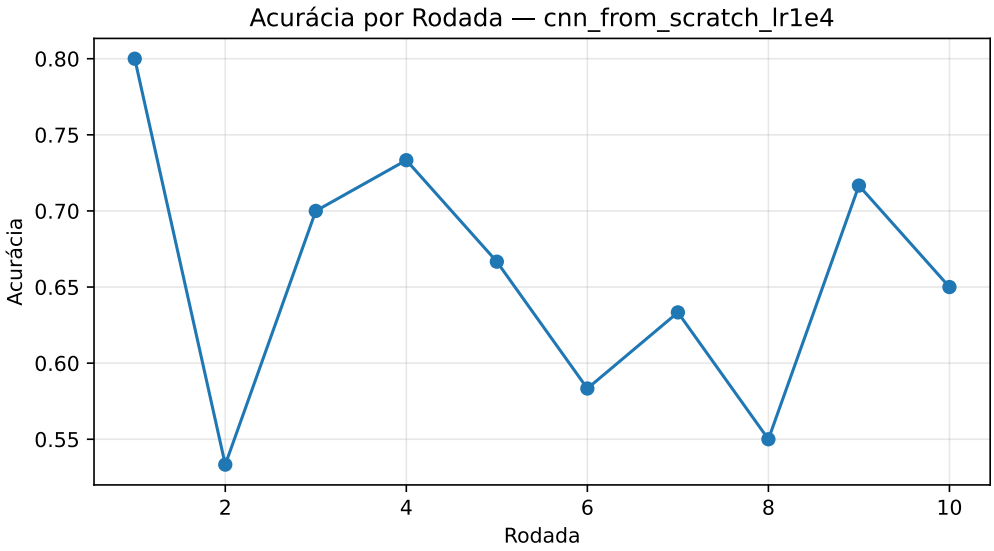

Figura 4: Acurácia por rodada — CNN do zero (lr $= 1 \times 10^{-4}$)

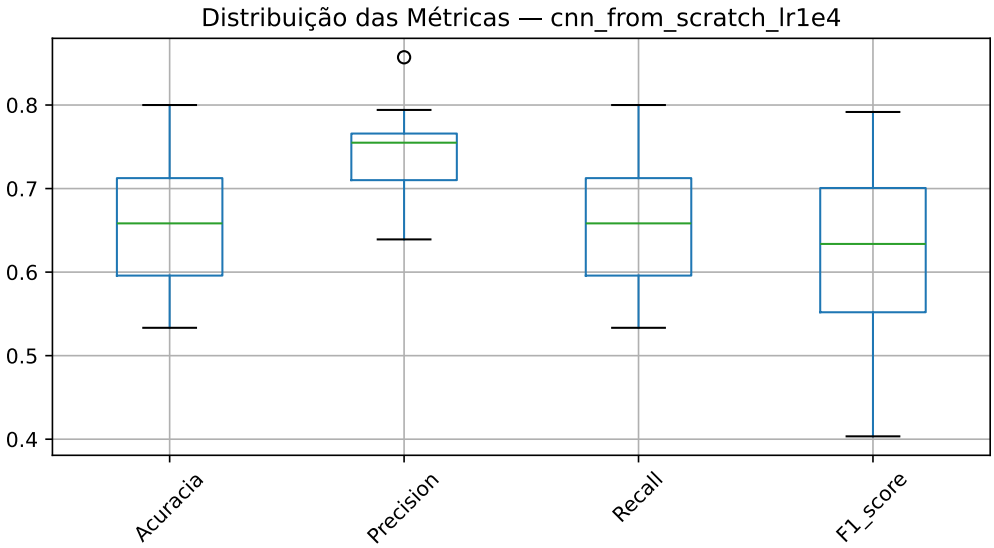

Figura 5: Boxplot das métricas — CNN do zero (lr $= 1 \times 10^{-4}$)

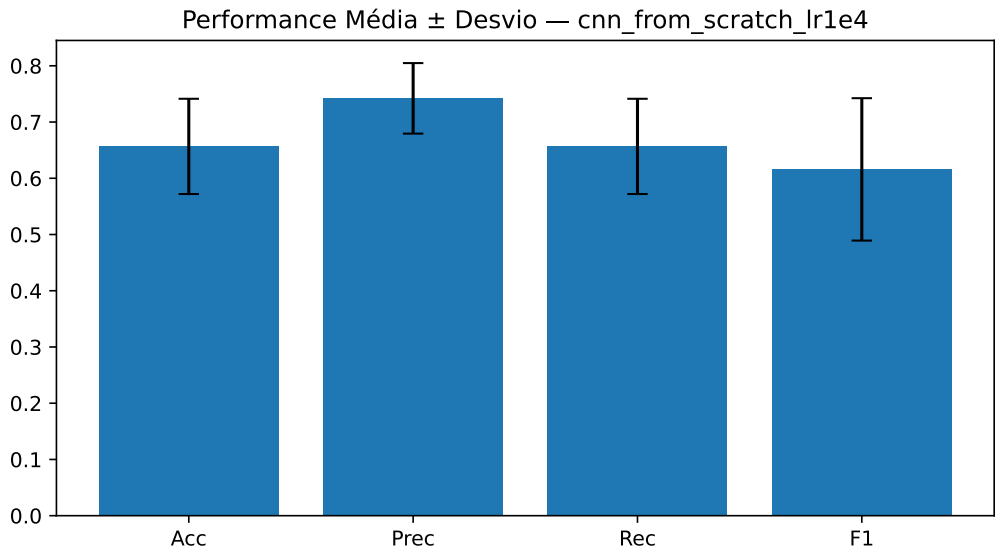

Figura 6: Média ± desvio padrão — CNN do zero (lr $= 1 \times 10^{-4}$)

5.6 Análise

Com lr $= 1 \times 10^{-4}$, a CNN do zero obteve acurácia média de 0,6567, **inferior** ao resultado com lr$= 5 \times 10^{-4}$ (0,7400). O coeficiente de variação foi de 12,9%, similar ao Modelo 1. Isso sugere que a taxa de aprendizado mais baixa pode ter dificultado a convergência dentro das 50 épocas com early stopping, ou levado o modelo a mínimos locais subótimos. Em ambos os casos de CNN do zero, o desempenho fica muito abaixo dos modelos de Transfer Learning.

6 Modelo 3 — MobileNetV2

6.1 Descrição do Backbone

O **MobileNetV2** [16] é uma arquitetura eficiente para dispositivos móveis, baseada em *inverted residuals* com conexões residuais em blocos de expansão linear. O operador principal é o *depthwise separable convolution*, que reduz drasticamente o número de parâmetros em relação a convoluções padrão. A versão utilizada é a original com entrada de 150×150.

6.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	MobileNetV2 (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$150 \times 150 \times 3$
Tamanho do Batch	32
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`mobilenet_v2.preprocess_input`

6.3 Resultados por Rodada

Tabela 9: Resultados por rodada — MobileNetV2

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
mobilenetv2	1	0	0.9833	0.9839	0.9833	0.9833	41.20	5.33	46.53
mobilenetv2	2	1	1.0000	1.0000	1.0000	1.0000	40.82	5.94	46.76
mobilenetv2	3	2	0.9333	0.9412	0.9333	0.9330	40.76	5.97	46.72
mobilenetv2	4	3	0.9500	0.9545	0.9500	0.9499	40.29	5.31	45.60
mobilenetv2	5	4	0.9667	0.9688	0.9667	0.9666	41.57	6.50	48.08
mobilenetv2	6	5	0.9833	0.9839	0.9833	0.9833	42.28	6.30	48.58
mobilenetv2	7	6	0.9667	0.9688	0.9667	0.9666	40.95	5.47	46.42
mobilenetv2	8	7	0.9500	0.9545	0.9500	0.9499	41.53	5.45	46.98
mobilenetv2	9	8	0.9833	0.9839	0.9833	0.9833	41.46	6.38	47.84
mobilenetv2	10	9	0.9833	0.9839	0.9833	0.9833	31.86	6.22	38.08

6.4 Estatísticas Resumo

Tabela 10: Resumo estatístico — MobileNetV2

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.9700	0.0205	0.0211	0.9553	0.9847
Precision	0.9723	0.0180	0.0185	0.9594	0.9852
Recall	0.9700	0.0205	0.0211	0.9553	0.9847
F1_score	0.9699	0.0206	0.0212	0.9552	0.9847
Tempo_Treino_s	40.2735	3.0055	0.0746	38.1236	42.4234
Tempo_Pred_s	5.8863	0.4626	0.0786	5.5554	6.2172
Tempo_Total_s	46.1598	2.9721	0.0644	44.0338	48.2857

6.5 Gráficos

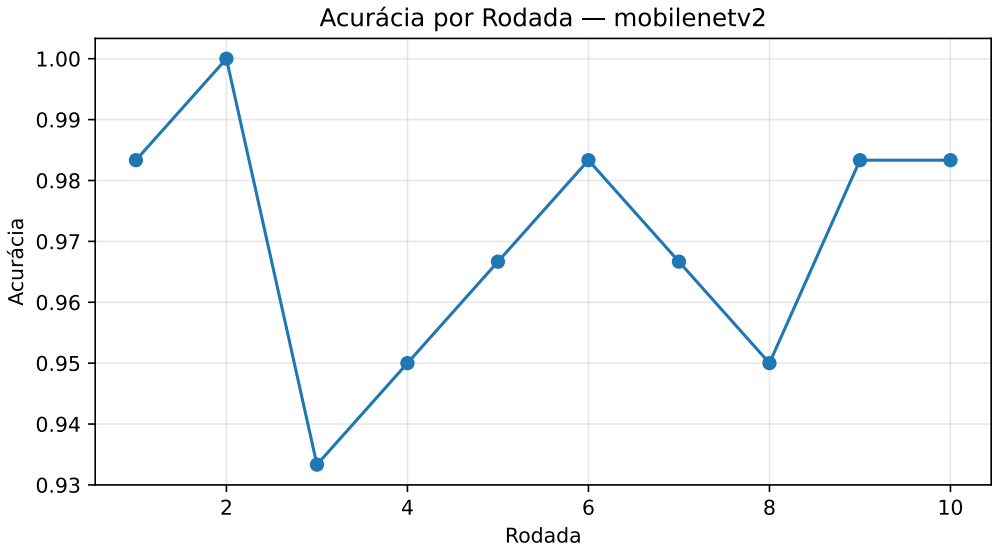

Figura 7: Acurácia por rodada — MobileNetV2

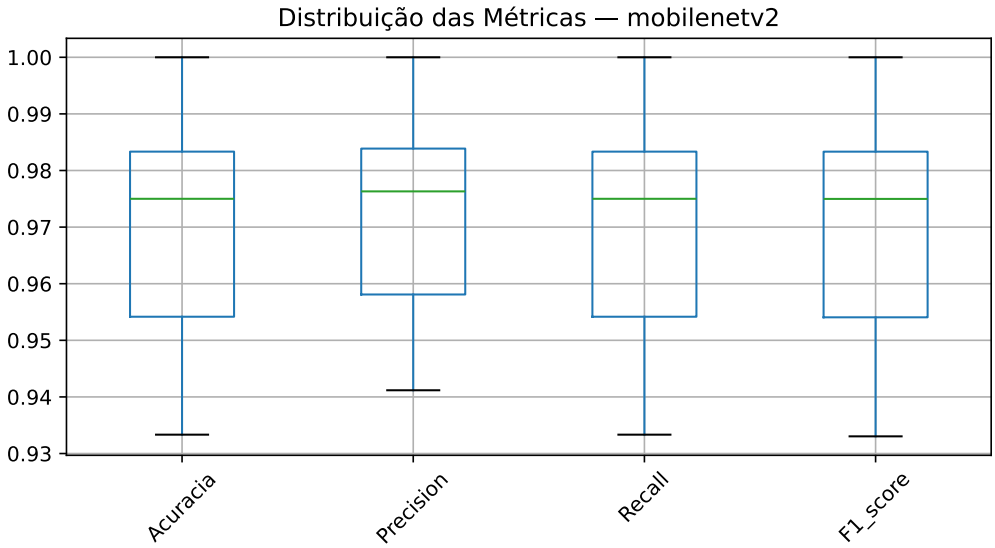

Figura 8: Boxplot das métricas — MobileNetV2

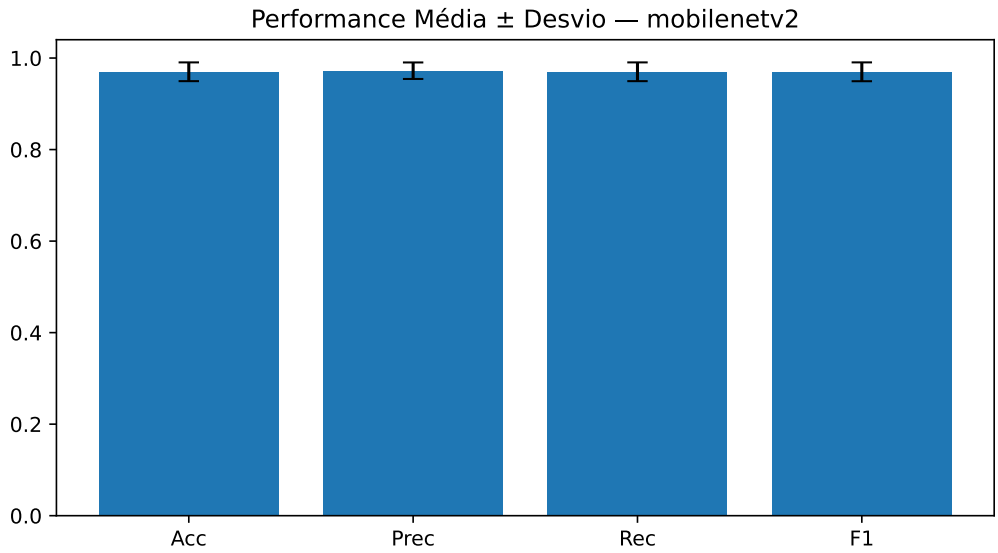

Figura 9: Média ± desvio padrão — MobileNetV2

6.6 Análise

O MobileNetV2 atingiu acurácia média de 0,9700 com desvio padrão de 0,0205 (CV = 2,1%), demonstrando estabilidade significativamente superior às CNNs do zero. O intervalo de confiança de 95% é [0,9553; 0,9847]. O tempo médio de treinamento foi de 40,3 s, tornando-o um dos modelos mais rápidos entre os de TL.

7 Modelo 4 — EfficientNetB0

7.1 Descrição do Backbone

O **EfficientNet** [17] introduz o conceito de *compound scaling*, escalando simultaneamente profundidade, largura e resolução da rede de forma balanceada. O **EfficientNetB0** é a versão base desta família, com resolução nativa de 224×224.

7.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	EfficientNetB0 (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$224 \times 224 \times 3$
Tamanho do Batch	32
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`efficientnet.preprocess_input`

7.3 Resultados por Rodada

Tabela 11: Resultados por rodada — EfficientNetB0

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
efficientnetb0	1	0	1.0000	1.0000	1.0000	1.0000	49.00	7.36	56.35
efficientnetb0	2	1	1.0000	1.0000	1.0000	1.0000	47.38	8.20	55.58
efficientnetb0	3	2	1.0000	1.0000	1.0000	1.0000	47.40	7.20	54.60
efficientnetb0	4	3	1.0000	1.0000	1.0000	1.0000	47.79	8.24	56.03
efficientnetb0	5	4	0.9833	0.9839	0.9833	0.9833	45.50	7.98	53.47
efficientnetb0	6	5	1.0000	1.0000	1.0000	1.0000	47.22	7.28	54.50
efficientnetb0	7	6	1.0000	1.0000	1.0000	1.0000	45.75	7.89	53.64
efficientnetb0	8	7	1.0000	1.0000	1.0000	1.0000	46.17	8.02	54.19
efficientnetb0	9	8	1.0000	1.0000	1.0000	1.0000	48.06	7.33	55.39
efficientnetb0	10	9	1.0000	1.0000	1.0000	1.0000	46.75	8.62	55.38

7.4 Estatísticas Resumo

Tabela 12: Resumo estatístico — EfficientNetB0

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.9983	0.0053	0.0053	0.9946	1.0021
Precision	0.9984	0.0051	0.0051	0.9947	1.0020
Recall	0.9983	0.0053	0.0053	0.9946	1.0021
F1_score	0.9983	0.0053	0.0053	0.9946	1.0021
Tempo_Treino_s	47.1019	1.0835	0.0230	46.3269	47.8770
Tempo_Pred_s	7.8116	0.4903	0.0628	7.4609	8.1623
Tempo_Total_s	54.9136	0.9837	0.0179	54.2099	55.6172

7.5 Gráficos

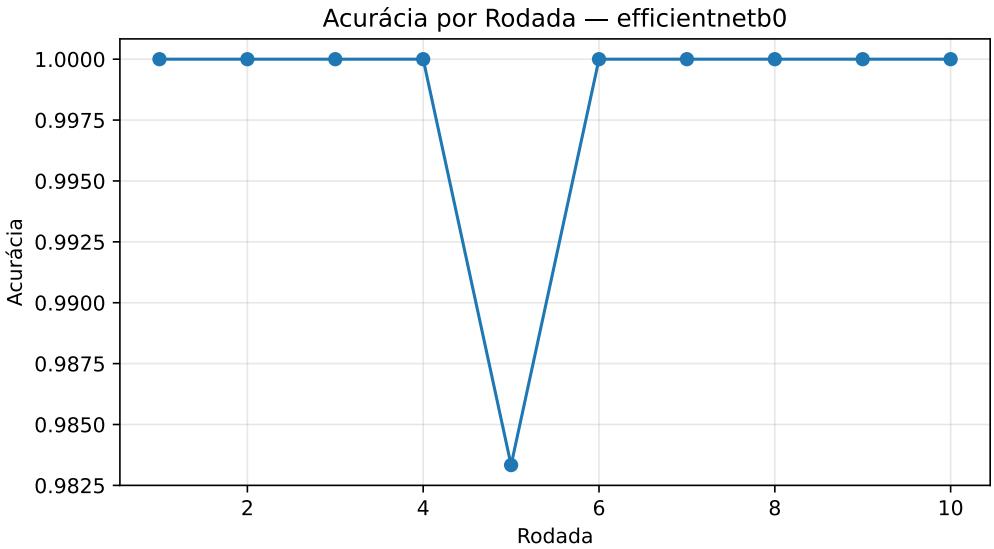

Figura 10: Acurácia por rodada — EfficientNetB0

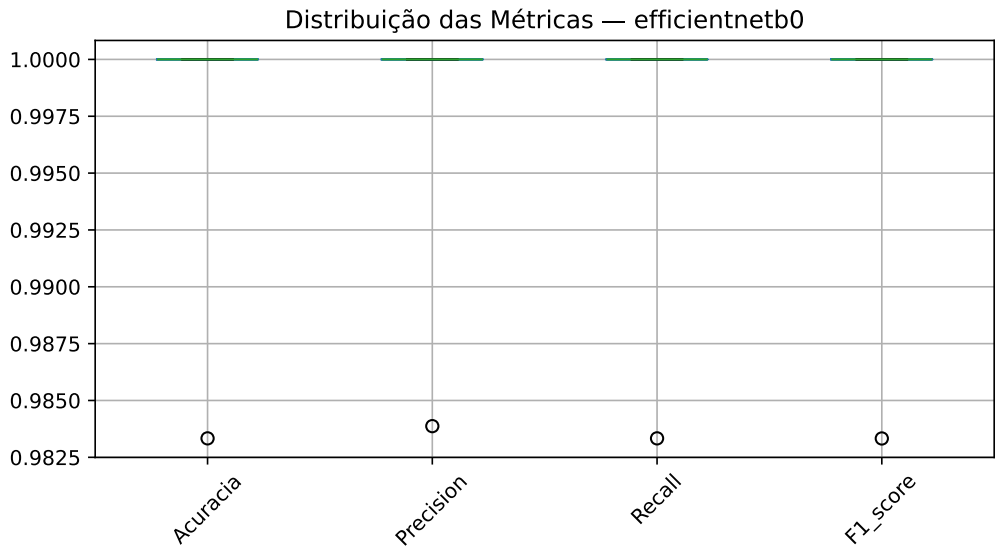

Figura 11: Boxplot das métricas — EfficientNetB0

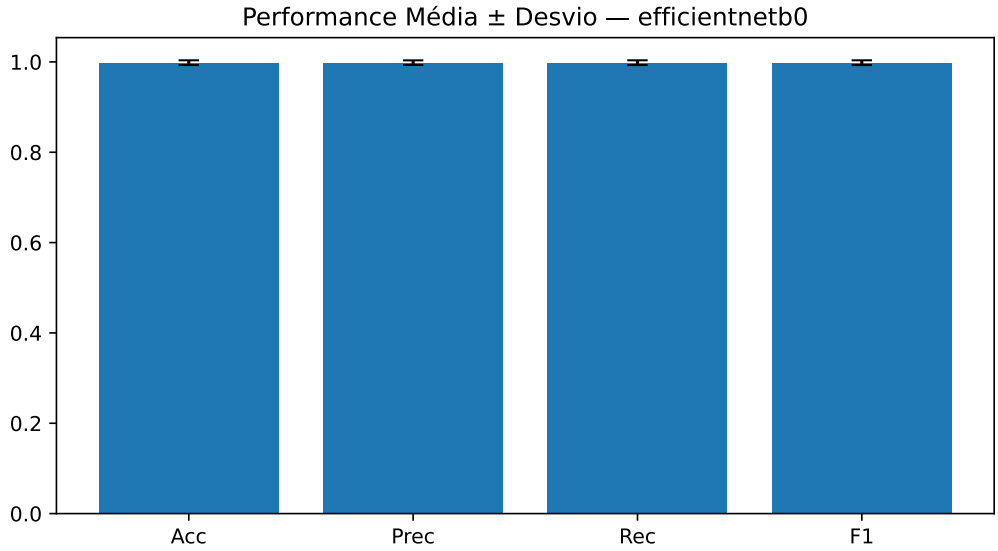

Figura 12: Média ± desvio padrão — EfficientNetB0

7.6 Análise

O EfficientNetB0 obteve acurácia média de 0,9983, com desvio padrão de apenas 0,0053 (CV = 0,5%), mostrando desempenho quase perfeito e alta estabilidade. O IC 95% é [0,9946; 1,0021]. O tempo médio de treinamento foi de 47,1 s, levemente superior ao MobileNetV2.

8 Modelo 5 — EfficientNetB1

8.1 Descrição do Backbone

O **EfficientNetB1** é a primeira escala acima do B0 na família EfficientNet, com resolução nativa de 240×240 e maior número de parâmetros. Mantém a mesma estratégia de *compound scaling*.

8.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	EfficientNetB1 (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$240 \times 240 \times 3$
Tamanho do Batch	32
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`efficientnet.preprocess_input`

8.3 Resultados por Rodada

Tabela 13: Resultados por rodada — EfficientNetB1

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
efficientnetb1	1	0	1.0000	1.0000	1.0000	1.0000	53.69	9.80	63.49
efficientnetb1	2	1	1.0000	1.0000	1.0000	1.0000	53.76	10.22	63.98
efficientnetb1	3	2	1.0000	1.0000	1.0000	1.0000	55.36	9.80	65.17
efficientnetb1	4	3	1.0000	1.0000	1.0000	1.0000	54.60	9.24	63.84
efficientnetb1	5	4	1.0000	1.0000	1.0000	1.0000	54.05	9.79	63.84
efficientnetb1	6	5	1.0000	1.0000	1.0000	1.0000	54.88	10.09	64.97
efficientnetb1	7	6	1.0000	1.0000	1.0000	1.0000	54.10	10.16	64.26
efficientnetb1	8	7	0.9833	0.9839	0.9833	0.9833	53.35	9.67	63.01
efficientnetb1	9	8	1.0000	1.0000	1.0000	1.0000	53.48	10.00	63.48
efficientnetb1	10	9	1.0000	1.0000	1.0000	1.0000	55.50	9.18	64.69

8.4 Estatísticas Resumo

Tabela 14: Resumo estatístico — EfficientNetB1

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.9983	0.0053	0.0053	0.9946	1.0021
Precision	0.9984	0.0051	0.0051	0.9947	1.0020
Recall	0.9983	0.0053	0.0053	0.9946	1.0021
F1_score	0.9983	0.0053	0.0053	0.9946	1.0021
Tempo_Treino_s	54.2768	0.7713	0.0142	53.7251	54.8285
Tempo_Pred_s	9.7962	0.3557	0.0363	9.5418	10.0506
Tempo_Total_s	64.0731	0.6945	0.0108	63.5763	64.5698

8.5 Gráficos

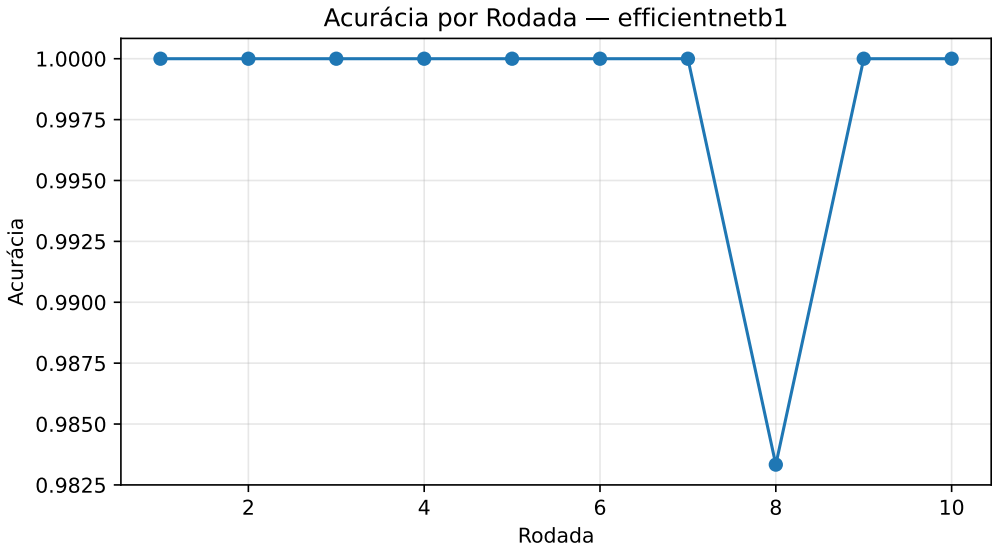

Figura 13: Acurácia por rodada — EfficientNetB1

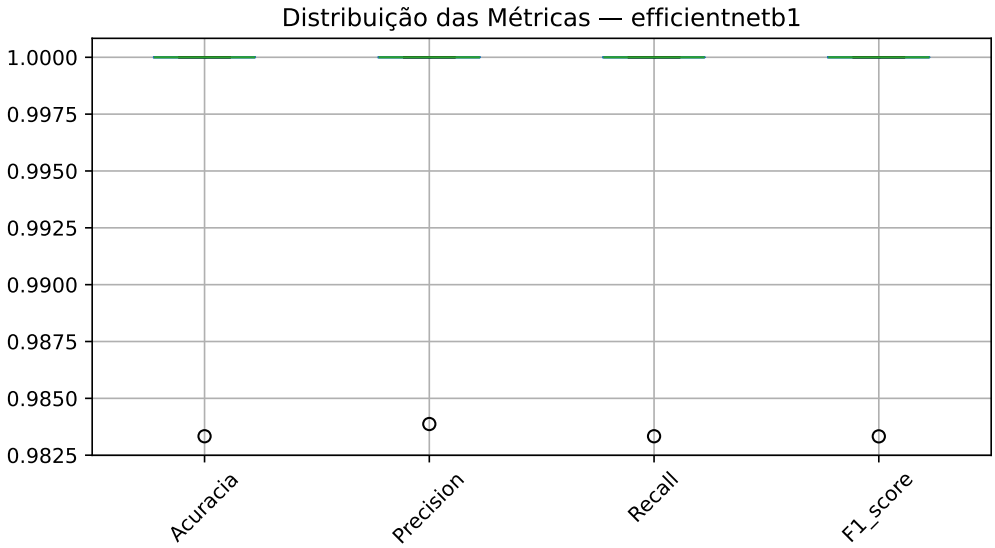

Figura 14: Boxplot das métricas — EfficientNetB1

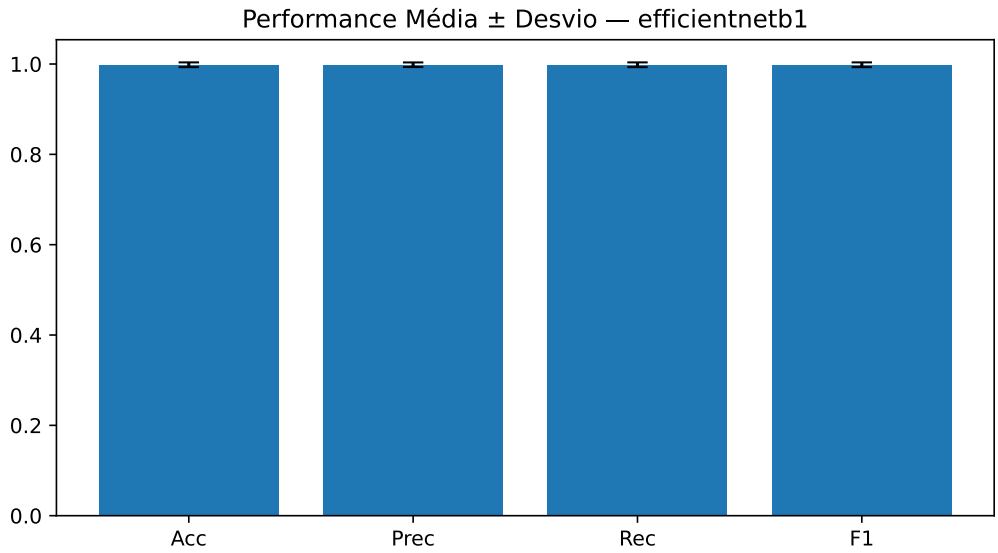

Figura 15: Média ± desvio padrão — EfficientNetB1

8.6 Análise

O EfficientNetB1 apresentou desempenho idêntico ao B0: acurácia média 0,9983 e desvio de 0,0053. O custo computacional foi ligeiramente maior (54,3 s de treinamento em média *vs.* 47,1 s do B0), sem ganho preditivo. Isso indica que o aumento de complexidade do B1 não agrega valor para esta tarefa binária.

9 Modelo 6 — EfficientNetB2

9.1 Descrição do Backbone

O **EfficientNetB2** continua o escalonamento da família EfficientNet, com resolução nativa de 260×260. É o maior dos três variantes avaliados neste trabalho.

9.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	EfficientNetB2 (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$260 \times 260 \times 3$
Tamanho do Batch	32
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`efficientnet.preprocess_input`

9.3 Resultados por Rodada

Tabela 15: Resultados por rodada — EfficientNetB2

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
efficientnetb2	1	0	1.0000	1.0000	1.0000	1.0000	55.58	10.13	65.70
efficientnetb2	2	1	1.0000	1.0000	1.0000	1.0000	55.95	9.54	65.50
efficientnetb2	3	2	1.0000	1.0000	1.0000	1.0000	56.39	10.14	66.53
efficientnetb2	4	3	1.0000	1.0000	1.0000	1.0000	55.45	10.02	65.47
efficientnetb2	5	4	1.0000	1.0000	1.0000	1.0000	55.75	9.75	65.50
efficientnetb2	6	5	1.0000	1.0000	1.0000	1.0000	54.56	10.15	64.70
efficientnetb2	7	6	1.0000	1.0000	1.0000	1.0000	57.39	9.46	66.85
efficientnetb2	8	7	1.0000	1.0000	1.0000	1.0000	56.06	10.67	66.73
efficientnetb2	9	8	1.0000	1.0000	1.0000	1.0000	54.01	9.99	64.00
efficientnetb2	10	9	1.0000	1.0000	1.0000	1.0000	56.04	10.09	66.13

9.4 Estatísticas Resumo

Tabela 16: Resumo estatístico — EfficientNetB2

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	1.0000	0.0000	0.0000	1.0000	1.0000
Precision	1.0000	0.0000	0.0000	1.0000	1.0000
Recall	1.0000	0.0000	0.0000	1.0000	1.0000
F1_score	1.0000	0.0000	0.0000	1.0000	1.0000
Tempo_Treino_s	55.7187	0.9358	0.0168	55.0493	56.3881
Tempo_Pred_s	9.9932	0.3444	0.0345	9.7469	10.2395
Tempo_Total_s	65.7119	0.8992	0.0137	65.0687	66.3551

9.5 Gráficos

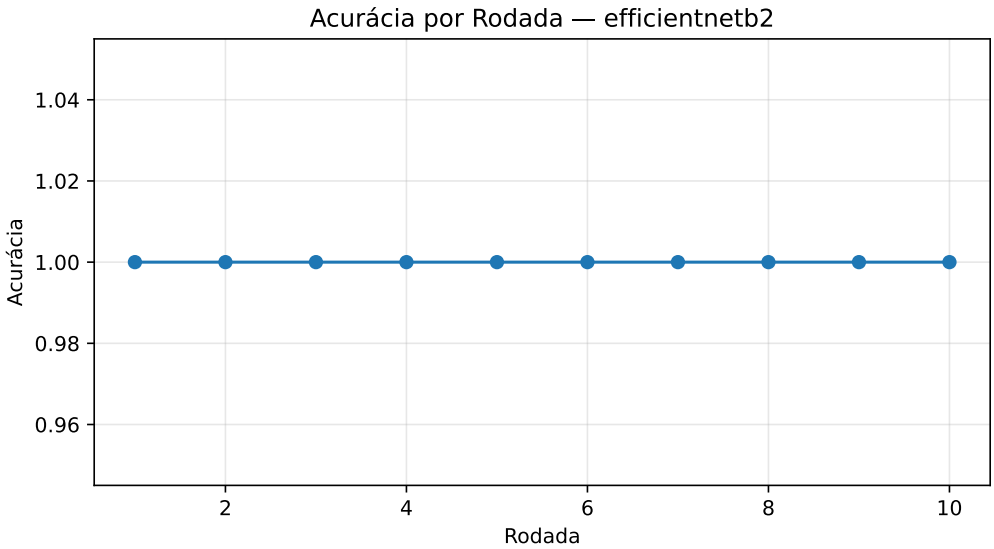

Figura 16: Acurácia por rodada — EfficientNetB2

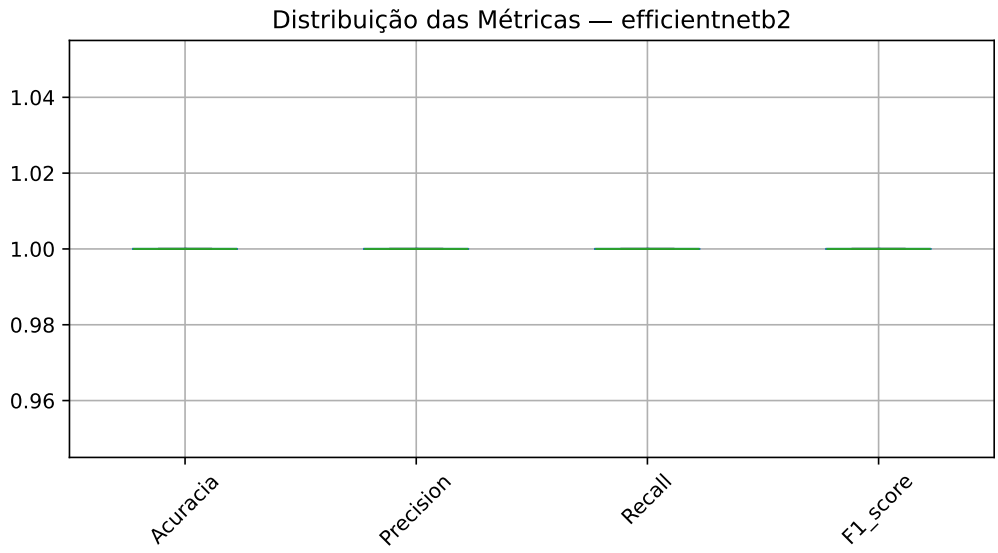

Figura 17: Boxplot das métricas — EfficientNetB2

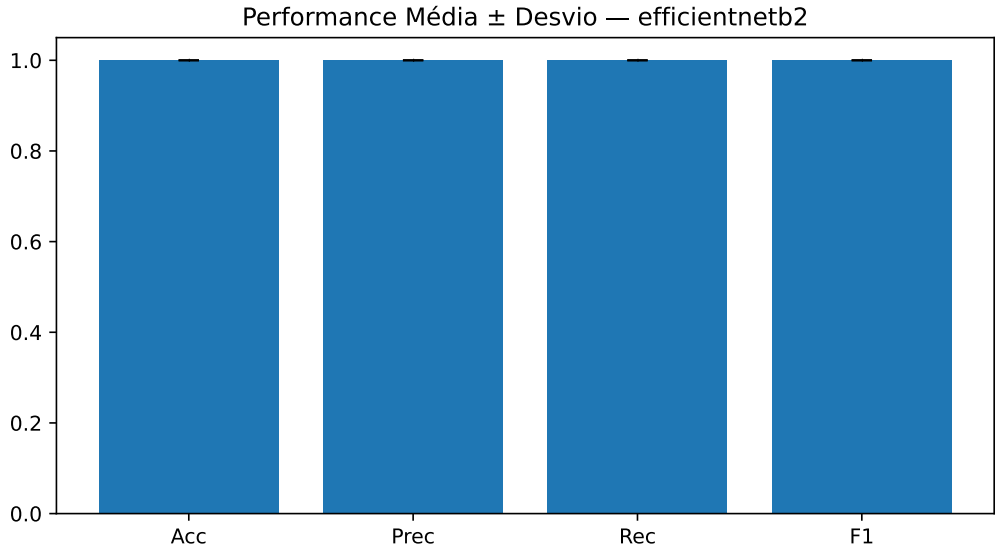

Figura 18: Média ± desvio padrão — EfficientNetB2

9.6 Análise

O EfficientNetB2 é um dos dois modelos com **desempenho perfeito**: acurácia, precisão, recall e F1-score médios iguais a 1,0000 com desvio padrão 0,0000 em todas as 10 rodadas. O treinamento demora em média 55,7 s, tornando-o mais lento que o B0 porém com resultado absolutamente estável.

10 Modelo 7 — ResNet50

10.1 Descrição do Backbone

A **ResNet50** [18] é uma rede residual com 50 camadas e conexões de atalho (*skip connections*) que resolvem o problema do gradiente desvanecente em redes profundas. Cada bloco residual aprende a diferença (resíduo) entre a entrada e a saída desejada.

10.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	ResNet50 (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$224 \times 224 \times 3$
Tamanho do Batch	32
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`resnet.preprocess_input`

10.3 Resultados por Rodada

Tabela 17: Resultados por rodada — ResNet50

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
resnet50	1	0	0.9500	0.9545	0.9500	0.9499	48.77	6.79	55.56
resnet50	2	1	0.9667	0.9688	0.9667	0.9666	48.83	7.30	56.13
resnet50	3	2	1.0000	1.0000	1.0000	1.0000	51.25	7.51	58.77
resnet50	4	3	0.9667	0.9688	0.9667	0.9666	48.13	11.95	60.07
resnet50	5	4	0.9000	0.9167	0.9000	0.8990	42.06	7.81	49.87
resnet50	6	5	0.9333	0.9412	0.9333	0.9330	49.07	7.95	57.02
resnet50	7	6	1.0000	1.0000	1.0000	1.0000	49.54	7.42	56.97
resnet50	8	7	1.0000	1.0000	1.0000	1.0000	45.89	7.75	53.64
resnet50	9	8	0.9833	0.9839	0.9833	0.9833	49.87	7.94	57.81
resnet50	10	9	0.9000	0.9167	0.9000	0.8990	50.41	7.52	57.93

10.4 Estatísticas Resumo

Tabela 18: Resumo estatístico — ResNet50

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.9600	0.0387	0.0403	0.9324	0.9876
Precision	0.9650	0.0323	0.0335	0.9419	0.9882
Recall	0.9600	0.0387	0.0403	0.9324	0.9876
F1_score	0.9597	0.0390	0.0407	0.9318	0.9877
Tempo_Treino_s	48.3813	2.6453	0.0547	46.4891	50.2736
Tempo_Pred_s	7.9943	1.4309	0.1790	6.9707	9.0178
Tempo_Total_s	56.3756	2.8893	0.0513	54.3089	58.4423

10.5 Gráficos

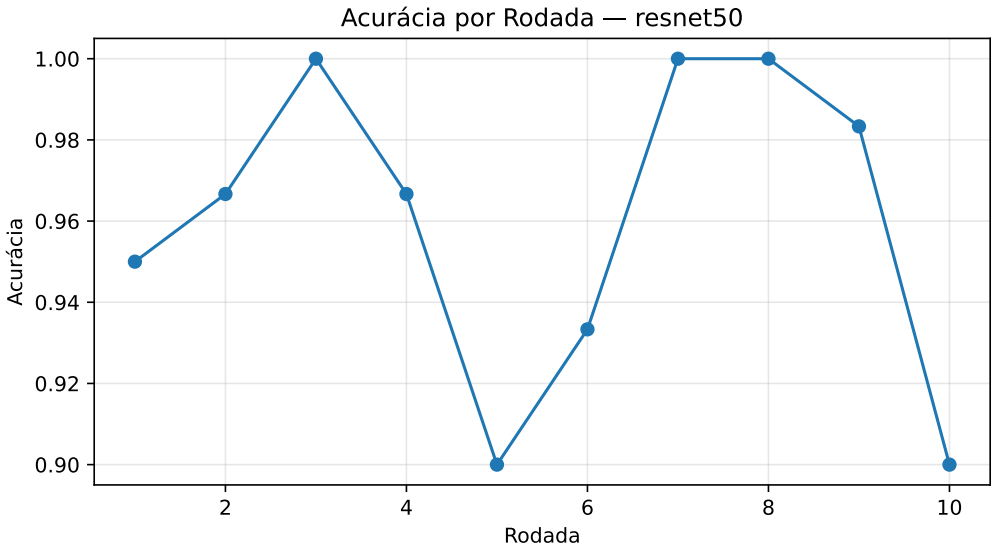

Figura 19: Acurácia por rodada — ResNet50

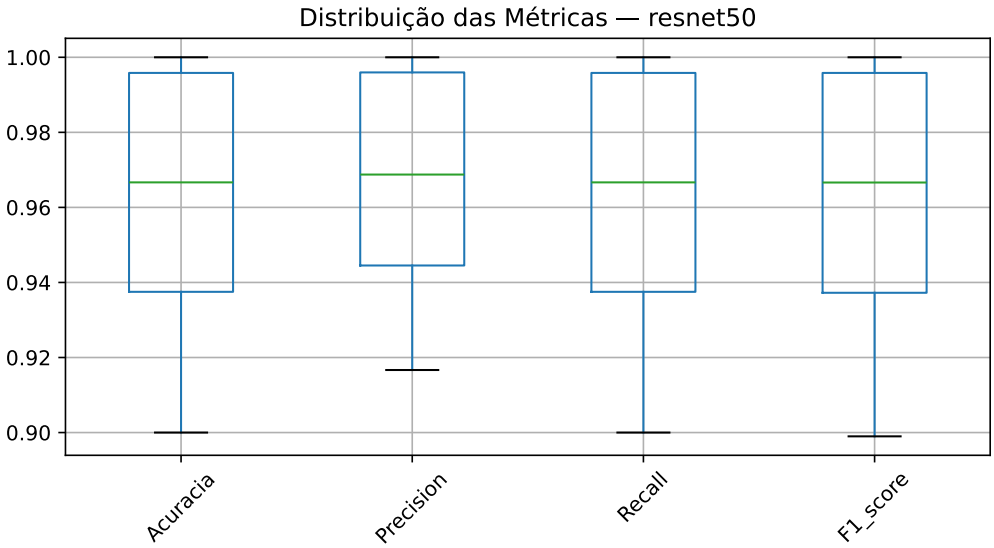

Figura 20: Boxplot das métricas — ResNet50

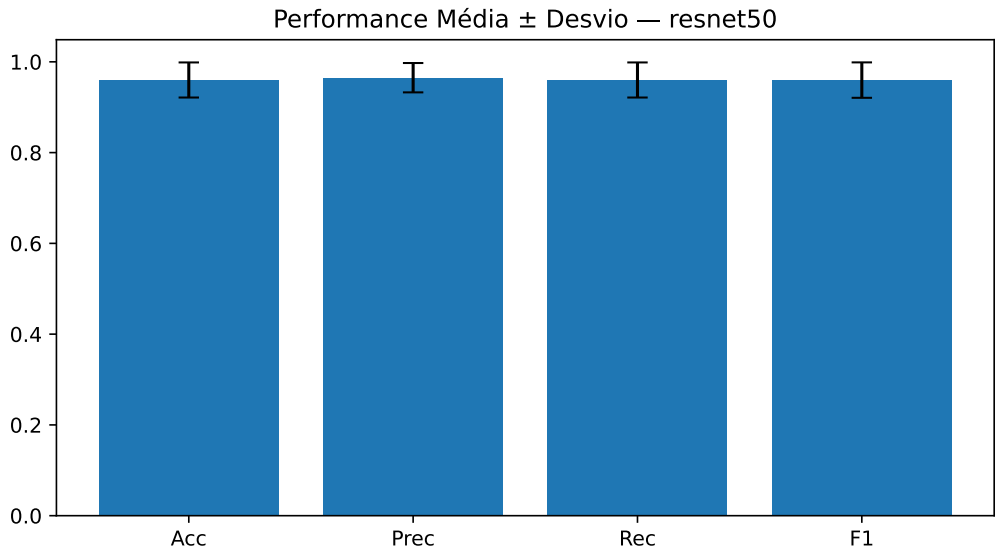

Figura 21: Média ± desvio padrão — ResNet50

10.6 Análise

A ResNet50 obteve acurácia média de 0,9600 com desvio de 0,0387 (CV = 4,0%). Embora seja um dos modelos de TL com menor desempenho médio neste experimento, permanece bem acima das CNNs do zero. A variabilidade moderada (IC 95%: [0,9324; 0,9876]) sugere sensibilidade à inicialização da cabeça de classificação. O tempo de treinamento (48,4 s) é similar ao EfficientNetB0.

11 Modelo 8 — DenseNet121

11.1 Descrição do Backbone

A **DenseNet121** [19] conecta cada camada a todas as camadas subsequentes (*dense connections*), promovendo reutilização de características e fluxo eficiente de gradientes. A DenseNet121 possui 121 camadas, organizadas em 4 blocos densos com camadas de transição entre eles.

11.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	DenseNet121 (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$224 \times 224 \times 3$
Tamanho do Batch	32
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`densenet.preprocess_input`

11.3 Resultados por Rodada

Tabela 19: Resultados por rodada — DenseNet121

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
densenet121	1	0	0.9833	0.9839	0.9833	0.9833	57.22	11.53	68.76
densenet121	2	1	1.0000	1.0000	1.0000	1.0000	54.36	11.44	65.81
densenet121	3	2	1.0000	1.0000	1.0000	1.0000	60.96	11.61	72.57
densenet121	4	3	0.9667	0.9688	0.9667	0.9666	56.94	11.18	68.13
densenet121	5	4	0.9833	0.9839	0.9833	0.9833	56.13	11.32	67.45
densenet121	6	5	0.9833	0.9839	0.9833	0.9833	58.71	11.34	70.04
densenet121	7	6	0.9667	0.9688	0.9667	0.9666	57.54	10.60	68.14
densenet121	8	7	0.9833	0.9839	0.9833	0.9833	55.59	11.38	66.97
densenet121	9	8	1.0000	1.0000	1.0000	1.0000	57.69	11.43	69.12
densenet121	10	9	0.9667	0.9688	0.9667	0.9666	57.11	11.23	68.34

11.4 Estatísticas Resumo

Tabela 20: Resumo estatístico — DenseNet121

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.9833	0.0136	0.0138	0.9736	0.9931
Precision	0.9842	0.0128	0.0130	0.9750	0.9933
Recall	0.9833	0.0136	0.0138	0.9736	0.9931
F1_score	0.9833	0.0136	0.0139	0.9736	0.9931
Tempo_Treino_s	57.2258	1.7831	0.0312	55.9503	58.5012
Tempo_Pred_s	11.3072	0.2795	0.0247	11.1073	11.5072
Tempo_Total_s	68.5330	1.8383	0.0268	67.2180	69.8480

11.5 Gráficos

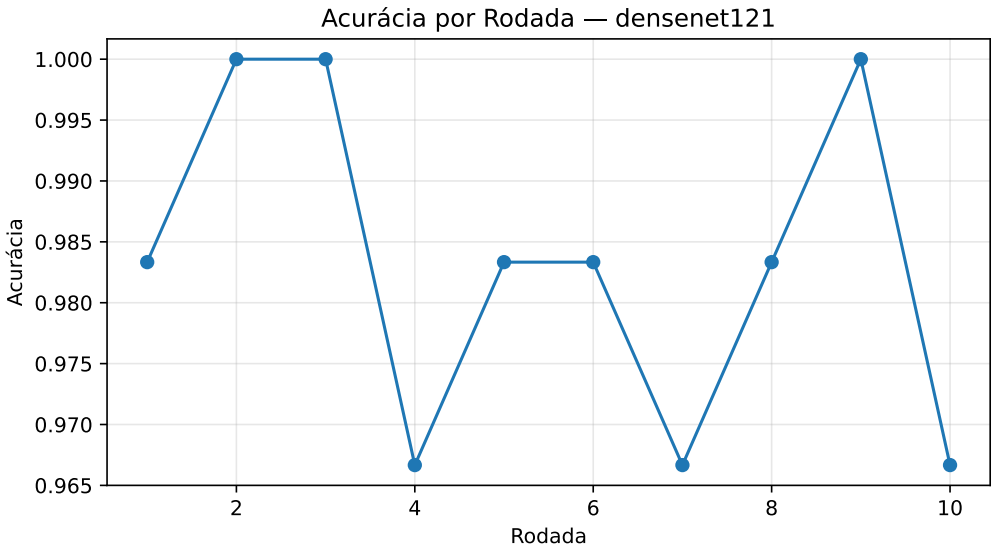

Figura 22: Acurácia por rodada — DenseNet121

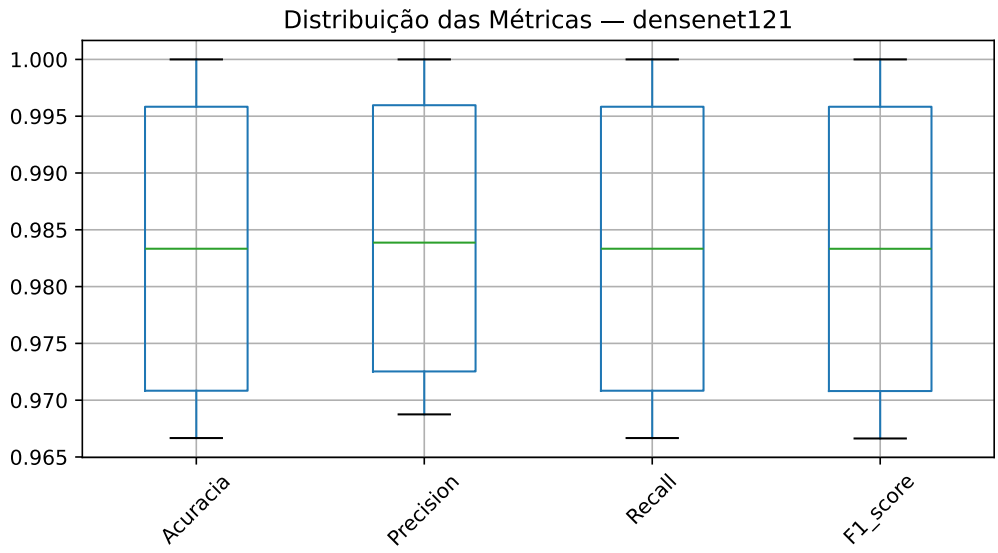

Figura 23: Boxplot das métricas — DenseNet121

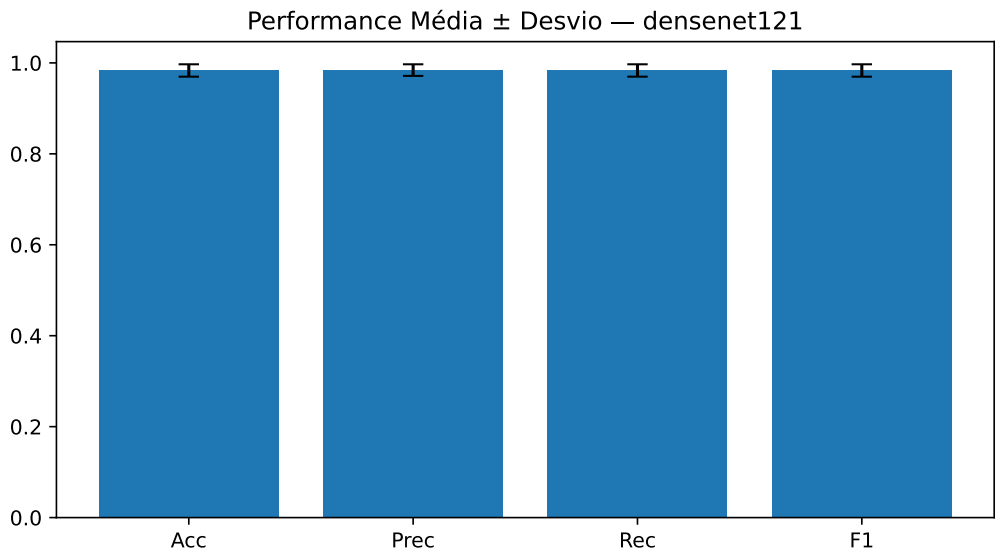

Figura 24: Média ± desvio padrão — DenseNet121

11.6 Análise

A DenseNet121 alcançou acurácia média de 0,9833 com desvio de 0,0136 (CV = 1,4%). O IC 95% é [0,9736; 0,9931], indicando alta estabilidade. O tempo médio de treinamento foi de 57,2 s, o que a torna um dos modelos mais lentos nessa resolução, reflexo do alto número de conexões densas internas.

12 Modelo 9 — MobileNetV3Small

12.1 Descrição do Backbone

O **MobileNetV3Small** [20] é uma versão compacta da família MobileNetV3, projetada para dispositivos de recursos muito limitados. Incorpora blocos *squeeze-and-excitation* e a função de ativação *hard-swish*. É a variante menor entre as duas versões do V3 avaliadas.

12.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	MobileNetV3Small (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$224 \times 224 \times 3$
Tamanho do Batch	32
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`mobilenet_v3.preprocess_input`

12.3 Resultados por Rodada

Tabela 21: Resultados por rodada — MobileNetV3Small

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
mobilenetv3small	1	0	0.8000	0.8571	0.8000	0.7917	41.12	6.35	47.47
mobilenetv3small	2	1	1.0000	1.0000	1.0000	1.0000	39.80	6.33	46.13
mobilenetv3small	3	2	1.0000	1.0000	1.0000	1.0000	39.26	5.49	44.75
mobilenetv3small	4	3	0.9833	0.9839	0.9833	0.9833	40.24	5.51	45.75
mobilenetv3small	5	4	1.0000	1.0000	1.0000	1.0000	40.92	5.59	46.52
mobilenetv3small	6	5	0.9833	0.9839	0.9833	0.9833	40.03	5.65	45.68
mobilenetv3small	7	6	0.9167	0.9286	0.9167	0.9161	40.69	5.78	46.48
mobilenetv3small	8	7	1.0000	1.0000	1.0000	1.0000	40.30	6.82	47.11
mobilenetv3small	9	8	0.9833	0.9839	0.9833	0.9833	39.85	6.00	45.85
mobilenetv3small	10	9	0.9833	0.9839	0.9833	0.9833	39.31	5.42	44.73

12.4 Estatísticas Resumo

Tabela 22: Resumo estatístico — MobileNetV3Small

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.9650	0.0631	0.0654	0.9199	1.0101
Precision	0.9721	0.0457	0.0470	0.9394	1.0048
Recall	0.9650	0.0631	0.0654	0.9199	1.0101
F1_score	0.9641	0.0656	0.0680	0.9172	1.0110
Tempo_Treino_s	40.1531	0.6306	0.0157	39.7021	40.6042
Tempo_Pred_s	5.8935	0.4665	0.0792	5.5598	6.2271
Tempo_Total_s	46.0466	0.8977	0.0195	45.4045	46.6887

12.5 Gráficos

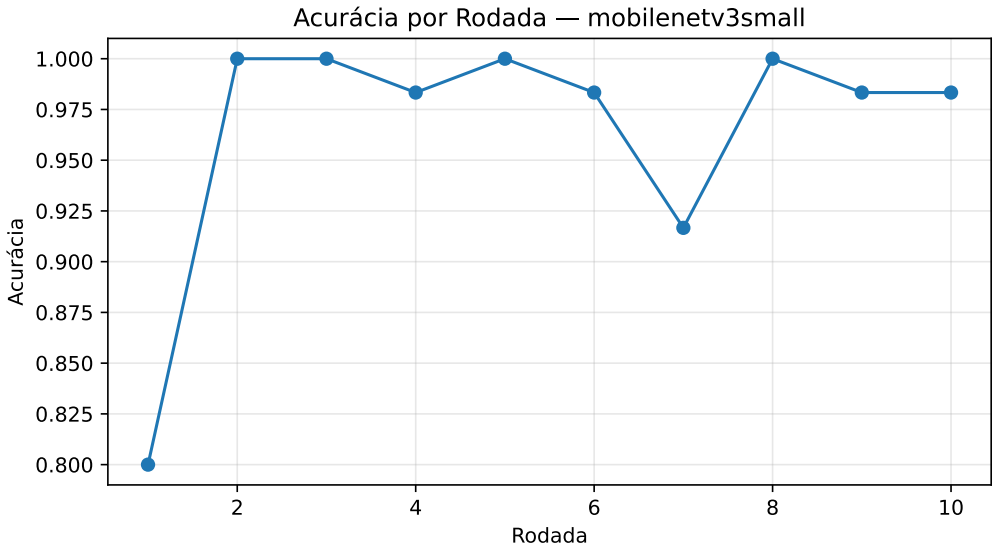

Figura 25: Acurácia por rodada — MobileNetV3Small

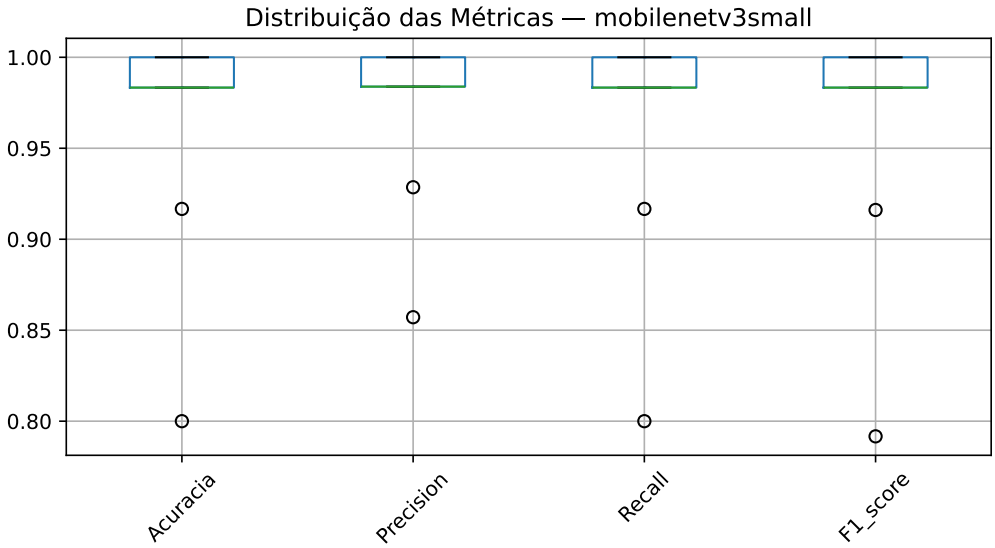

Figura 26: Boxplot das métricas — MobileNetV3Small

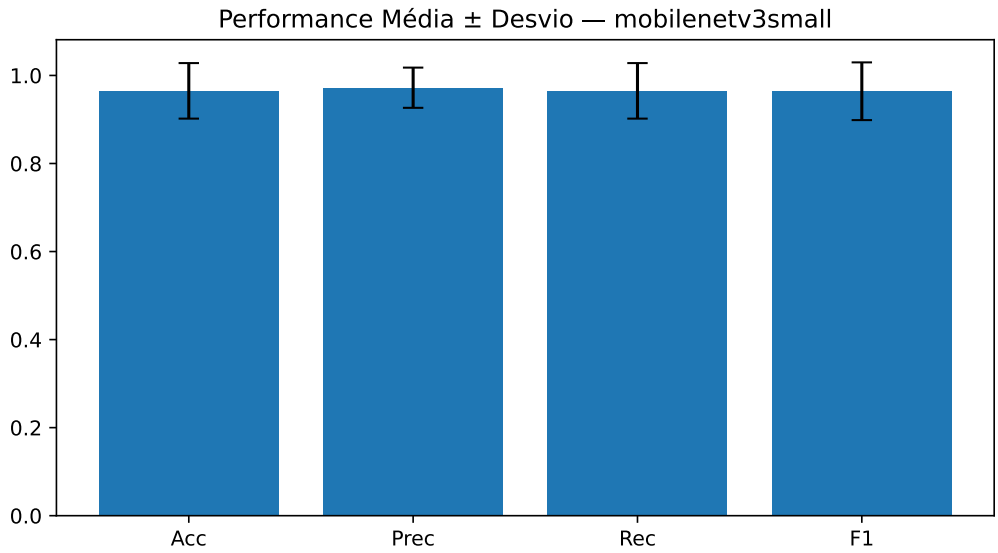

Figura 27: Média ± desvio padrão — MobileNetV3Small

12.6 Análise

O MobileNetV3Small atingiu acurácia média de 0,9650 com desvio de 0,0631 (CV = 6,5%), sendo o modelo de TL com maior variabilidade neste experimento. O IC 95% de [0,9199; 1,0101] ultrapassa 1,0 no limite superior (artefato estatístico aceitável). Apesar de ser o modelo mais compacto e rápido do grupo V3, sua menor capacidade representacional resulta em instabilidade para esta tarefa.

13 Modelo 10 — MobileNetV3Large

13.1 Descrição do Backbone

O **MobileNetV3Large** é a versão de maior capacidade da família MobileNetV3, com mais filtros e estágios que o *Small*. Mantém a eficiência para hardware limitado, mas com melhor desempenho em tarefas mais complexas.

13.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	MobileNetV3Large (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$224 \times 224 \times 3$
Tamanho do Batch	32
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`mobilenet_v3.preprocess_input`

13.3 Resultados por Rodada

Tabela 23: Resultados por rodada — MobileNetV3Large

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
mobilenetv3large	1	0	0.9333	0.9412	0.9333	0.9330	43.20	6.82	50.02
mobilenetv3large	2	1	0.9667	0.9667	0.9667	0.9667	42.15	6.84	48.99
mobilenetv3large	3	2	0.9833	0.9839	0.9833	0.9833	43.15	6.07	49.22
mobilenetv3large	4	3	0.9833	0.9839	0.9833	0.9833	41.98	5.92	47.90
mobilenetv3large	5	4	0.9833	0.9839	0.9833	0.9833	42.00	6.79	48.79
mobilenetv3large	6	5	1.0000	1.0000	1.0000	1.0000	41.74	6.95	48.69
mobilenetv3large	7	6	1.0000	1.0000	1.0000	1.0000	42.49	6.94	49.43
mobilenetv3large	8	7	0.9833	0.9839	0.9833	0.9833	44.28	5.96	50.24
mobilenetv3large	9	8	0.9000	0.9072	0.9000	0.8996	43.29	6.89	50.19
mobilenetv3large	10	9	0.9667	0.9667	0.9667	0.9667	42.42	6.47	48.89

13.4 Estatísticas Resumo

Tabela 24: Resumo estatístico — MobileNetV3Large

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.9700	0.0312	0.0322	0.9477	0.9923
Precision	0.9717	0.0285	0.0294	0.9513	0.9921
Recall	0.9700	0.0312	0.0322	0.9477	0.9923
F1_score	0.9699	0.0314	0.0324	0.9475	0.9924
Tempo_Treino_s	42.6697	0.7924	0.0186	42.1029	43.2365
Tempo_Pred_s	6.5665	0.4238	0.0645	6.2633	6.8696
Tempo_Total_s	49.2361	0.7459	0.0151	48.7026	49.7697

13.5 Gráficos

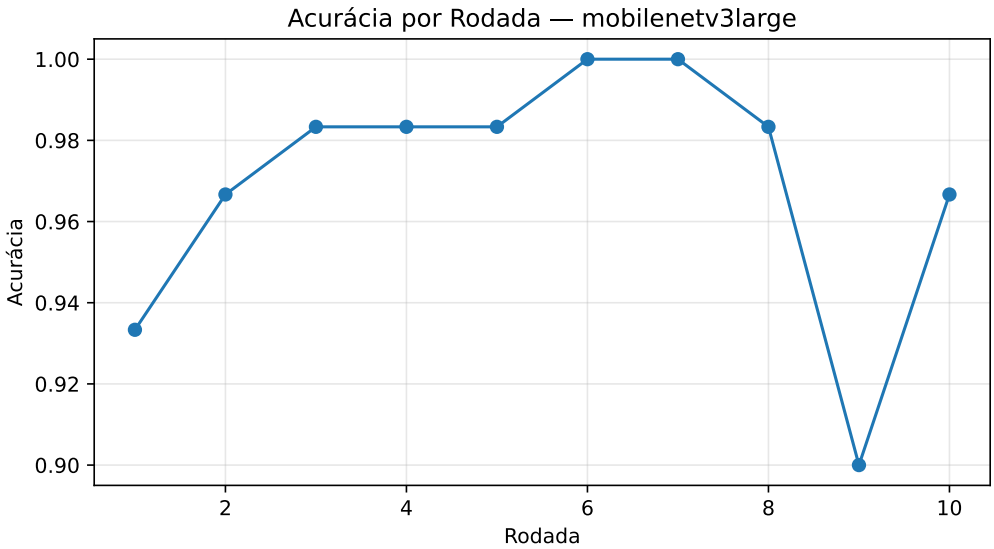

Figura 28: Acurácia por rodada — MobileNetV3Large

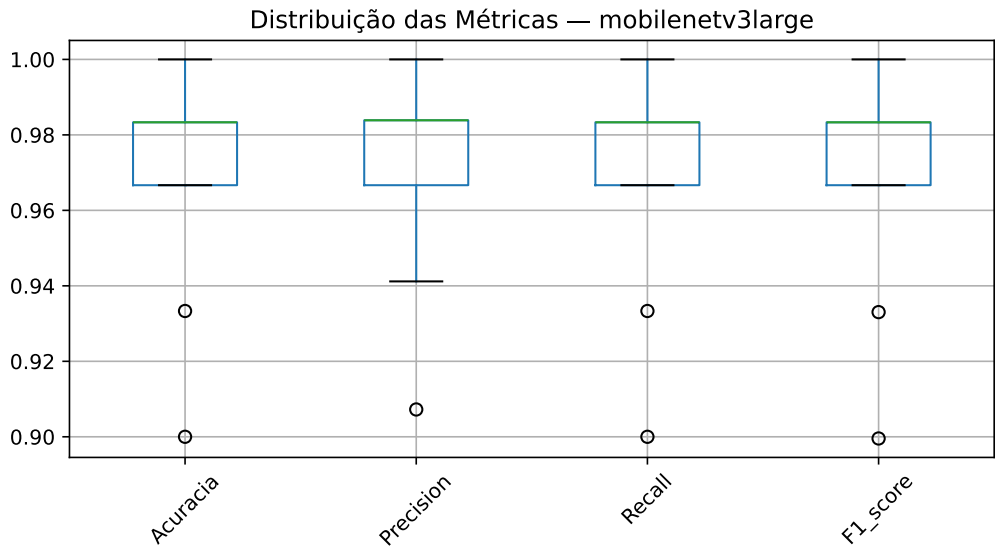

Figura 29: Boxplot das métricas — MobileNetV3Large

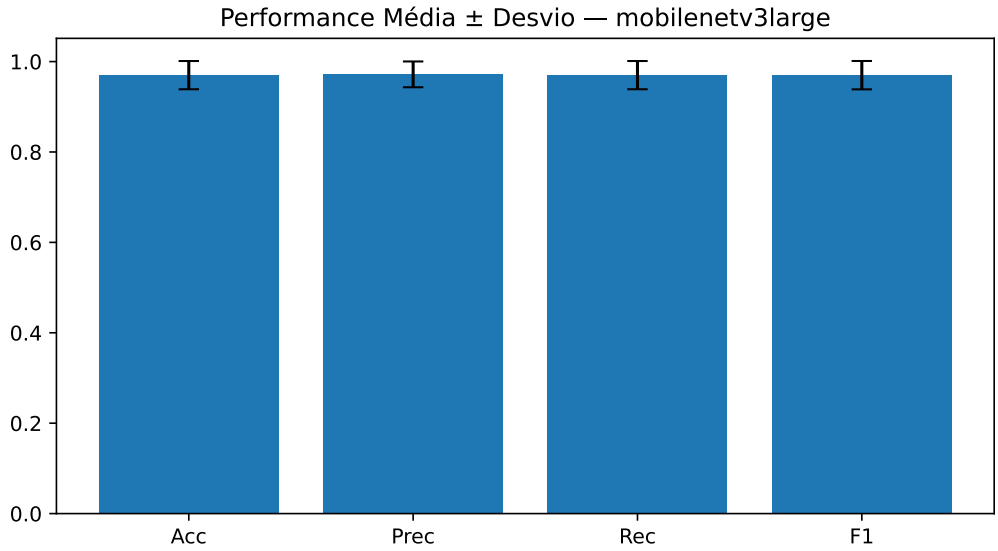

Figura 30: Média ± desvio padrão — MobileNetV3Large

13.6 Análise

O MobileNetV3Large melhorou em relação ao Small: acurácia média de 0,9700 com desvio de 0,0312 (CV = 3,2%). O IC 95% é [0,9477; 0,9923]. O tempo de treinamento (42,7 s) é similar ao MobileNetV2. A versão Large do V3 mostra melhor equilíbrio entre estabilidade e eficiência comparado à versão Small.

14 Modelo 11 — NASNetMobile

14.1 Descrição do Backbone

O **NASNetMobile** [21] é uma arquitetura obtida por *Neural Architecture Search* (NAS), em que o design da rede é automaticamente otimizado por um algoritmo de busca. A versão *Mobile* é compacta, balanceando desempenho e eficiência.

14.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	NASNetMobile (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$224 \times 224 \times 3$
Tamanho do Batch	32
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`nasnet.preprocess_input`

14.3 Resultados por Rodada

Tabela 25: Resultados por rodada — NASNetMobile

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
nasnetmobile	1	0	0.9333	0.9412	0.9333	0.9330	67.15	15.30	82.45
nasnetmobile	2	1	0.9667	0.9667	0.9667	0.9667	66.48	15.15	81.63
nasnetmobile	3	2	0.9333	0.9353	0.9333	0.9333	68.40	15.12	83.51
nasnetmobile	4	3	0.9833	0.9839	0.9833	0.9833	65.00	15.19	80.19
nasnetmobile	5	4	1.0000	1.0000	1.0000	1.0000	66.90	15.34	82.25
nasnetmobile	6	5	0.9833	0.9839	0.9833	0.9833	65.30	15.11	80.42
nasnetmobile	7	6	1.0000	1.0000	1.0000	1.0000	63.09	15.14	78.22
nasnetmobile	8	7	0.9500	0.9545	0.9500	0.9499	73.01	15.00	88.01
nasnetmobile	9	8	0.9833	0.9839	0.9833	0.9833	67.04	15.64	82.68
nasnetmobile	10	9	0.9500	0.9505	0.9500	0.9500	69.05	15.55	84.60

14.4 Estatísticas Resumo

Tabela 26: Resumo estatístico — NASNetMobile

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.9683	0.0254	0.0262	0.9502	0.9865
Precision	0.9700	0.0237	0.0244	0.9530	0.9869
Recall	0.9683	0.0254	0.0262	0.9502	0.9865
F1_score	0.9683	0.0255	0.0263	0.9501	0.9865
Tempo_Treino_s	67.1419	2.6777	0.0399	65.2265	69.0573
Tempo_Pred_s	15.2543	0.2042	0.0134	15.1082	15.4004
Tempo_Total_s	82.3962	2.6775	0.0325	80.4810	84.3114

14.5 Gráficos

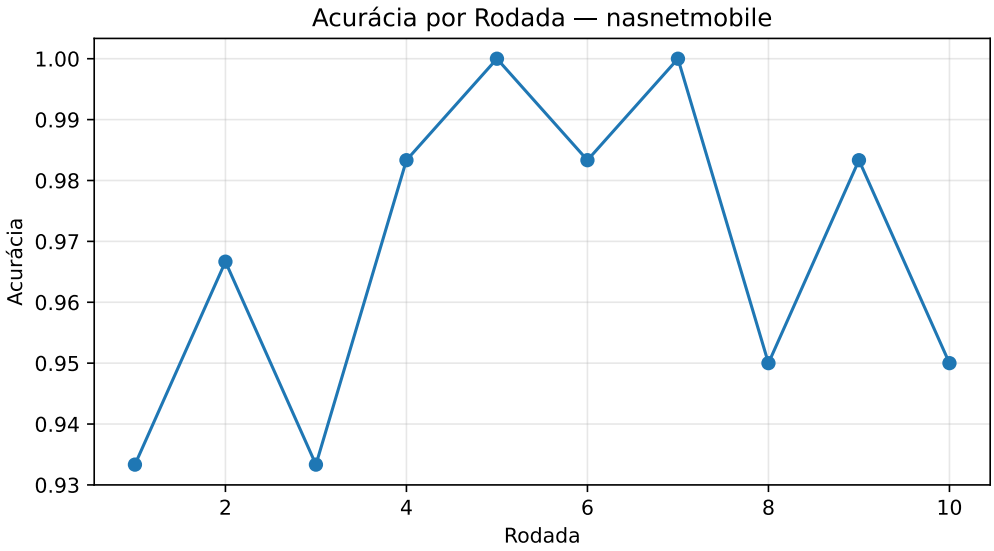

Figura 31: Acurácia por rodada — NASNetMobile

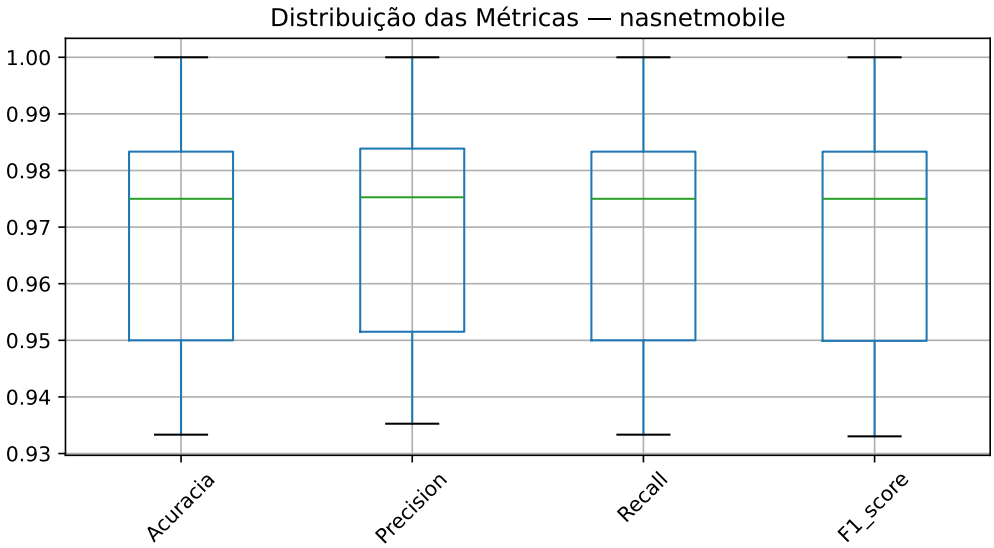

Figura 32: Boxplot das métricas — NASNetMobile

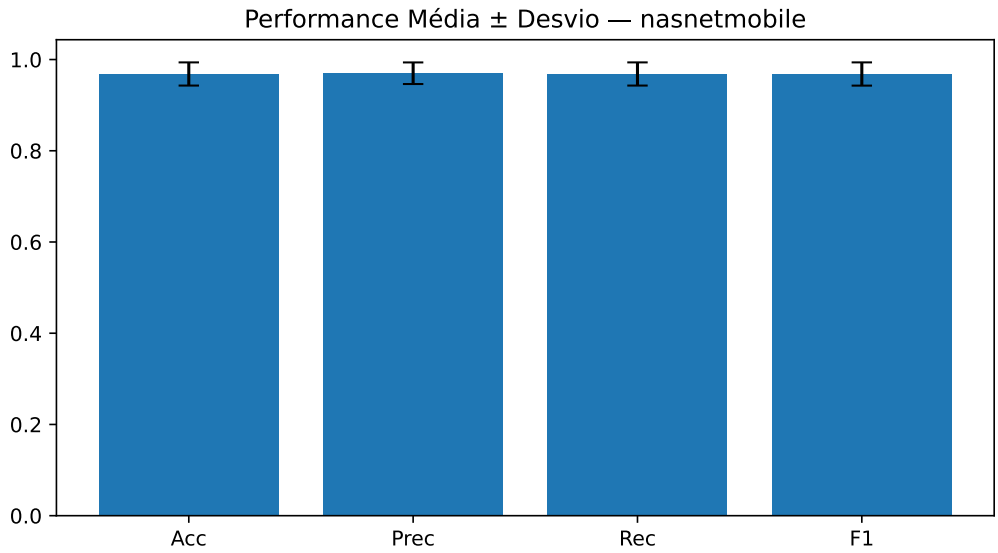

Figura 33: Média ± desvio padrão — NASNetMobile

14.6 Análise

O NASNetMobile obteve acurácia média de 0,9683 com desvio de 0,0254 (CV = 2,6%), com IC 95% em [0,9502; 0,9865]. Destaca-se negativamente pelo maior tempo de predição (15,3 s), consequência da topologia complexa gerada pelo NAS, que aumenta a latência de inferência. Seu tempo total médio (82,4 s) é um dos mais elevados entre os modelos leves.

15 Modelo 12 — InceptionV3

15.1 Descrição do Backbone

A **InceptionV3** [22] utiliza módulos Inception que paralelizam convoluções de diferentes tamanhos (1×1, 3×3, 5×5) e as concatenam. O V3 introduz fatorização de convoluções e normalização de batch, exigindo entrada mínima de 299×299.

15.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	InceptionV3 (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$299 \times 299 \times 3$
Tamanho do Batch	16
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`inception_v3.preprocess_input`

15.3 Resultados por Rodada

Tabela 27: Resultados por rodada — InceptionV3

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
inceptionv3	1	0	1.0000	1.0000	1.0000	1.0000	90.67	8.64	99.31
inceptionv3	2	1	0.9833	0.9839	0.9833	0.9833	106.61	8.01	114.62
inceptionv3	3	2	1.0000	1.0000	1.0000	1.0000	96.09	8.45	104.54
inceptionv3	4	3	0.9833	0.9839	0.9833	0.9833	106.56	7.34	113.91
inceptionv3	5	4	0.9667	0.9688	0.9667	0.9666	104.99	8.04	113.03
inceptionv3	6	5	0.9667	0.9688	0.9667	0.9666	106.95	8.58	115.53
inceptionv3	7	6	0.9833	0.9839	0.9833	0.9833	106.10	7.75	113.85
inceptionv3	8	7	0.9833	0.9839	0.9833	0.9833	103.22	7.34	110.55
inceptionv3	9	8	0.9833	0.9839	0.9833	0.9833	91.46	7.90	99.35
inceptionv3	10	9	0.9833	0.9839	0.9833	0.9833	79.42	8.30	87.73

15.4 Estatísticas Resumo

Tabela 28: Resumo estatístico — InceptionV3

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	0.9833	0.0111	0.0113	0.9754	0.9913
Precision	0.9841	0.0104	0.0106	0.9766	0.9915
Recall	0.9833	0.0111	0.0113	0.9754	0.9913
F1_score	0.9833	0.0111	0.0113	0.9754	0.9913
Tempo_Treino_s	99.2064	9.4282	0.0950	92.4623	105.9504
Tempo_Pred_s	8.0358	0.4683	0.0583	7.7008	8.3707
Tempo_Total_s	107.2421	9.2435	0.0862	100.6302	113.8541

15.5 Gráficos

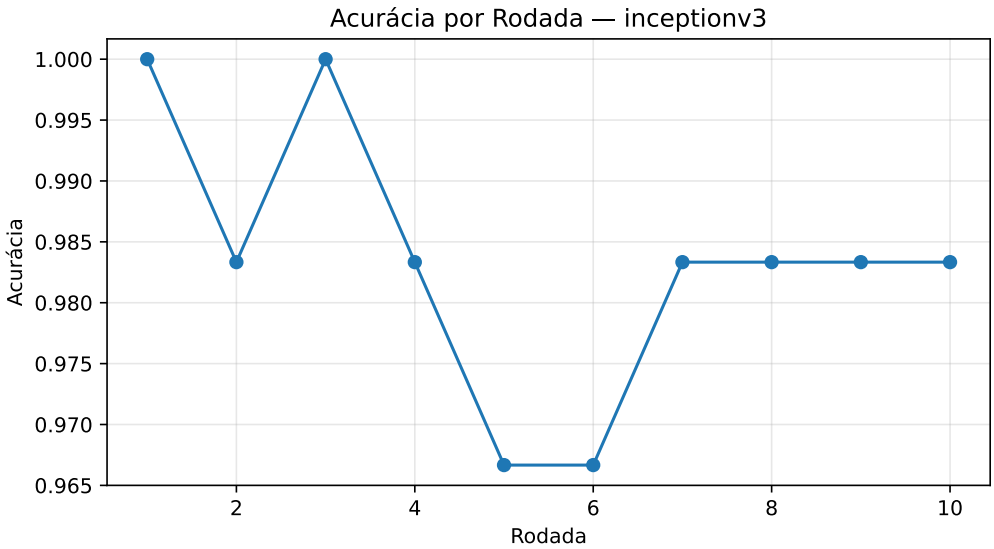

Figura 34: Acurácia por rodada — InceptionV3

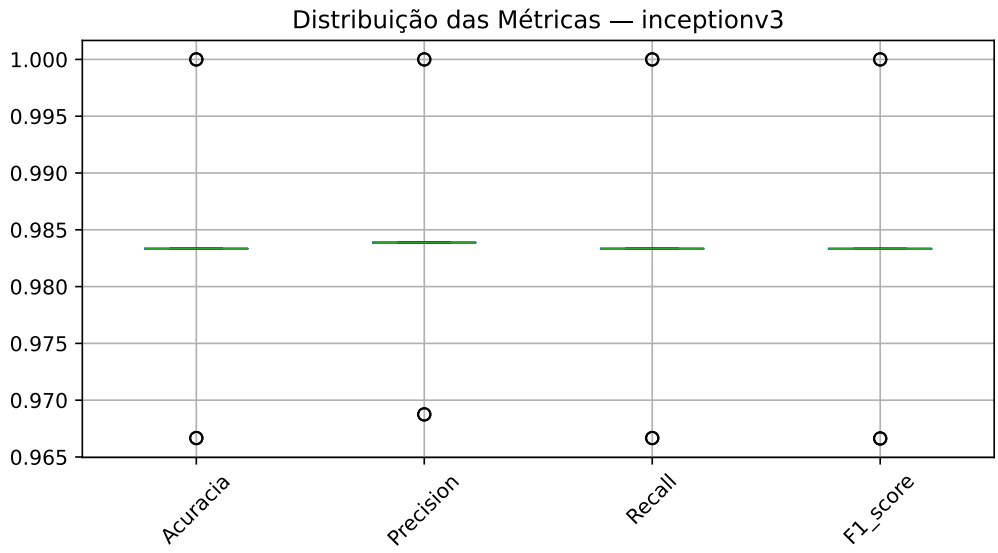

Figura 35: Boxplot das métricas — InceptionV3

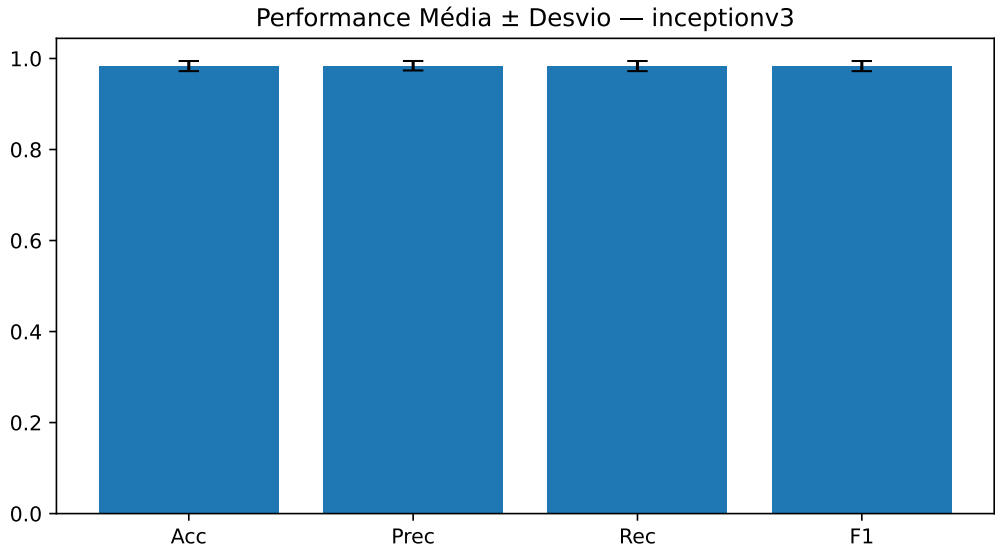

Figura 36: Média ± desvio padrão — InceptionV3

15.6 Análise

A InceptionV3 obteve acurácia média de 0,9833 com desvio de 0,0111 (CV = 1,1%), com IC 95% em [0,9754; 0,9913]. Apresenta alto desempenho preditivo, porém com o maior tempo de treinamento entre os modelos de resolução 299 × 299, chegando a 99,2 s em média. O batch de 16 (necessário pela resolução maior) contribui para esse custo.

16 Modelo 13 — Xception

16.1 Descrição do Backbone

A **Xception** [23] é uma extensão da Inception que substitui os módulos Inception por *depthwise separable convolutions* em todas as camadas. O nome vem de *Extreme Inception*. Assim como a InceptionV3, requer entrada de 299×299.

16.2 Hiperparâmetros

Parâmetro	**Valor**
Backbone	Xception (ImageNet, congelado)
Taxa de Aprendizado	1×10^{-4}
Imagem de Entrada	$299 \times 299 \times 3$
Tamanho do Batch	16
Épocas Máximas	30
Otimizador	Adam
Pré-processamento	`xception.preprocess_input`

16.3 Resultados por Rodada

Tabela 29: Resultados por rodada — Xception

Modelo	Rodada	Seed	Acuracia	Precision	Recall	F1_score	Tempo_Treino_s	Tempo_Pred_s	Tempo_Total_s
xception	1	0	1.0000	1.0000	1.0000	1.0000	110.03	5.23	115.25
xception	2	1	1.0000	1.0000	1.0000	1.0000	114.53	6.44	120.97
xception	3	2	1.0000	1.0000	1.0000	1.0000	112.82	6.08	118.90
xception	4	3	1.0000	1.0000	1.0000	1.0000	111.19	5.31	116.51
xception	5	4	1.0000	1.0000	1.0000	1.0000	110.32	5.26	115.58
xception	6	5	1.0000	1.0000	1.0000	1.0000	108.32	6.02	114.34
xception	7	6	1.0000	1.0000	1.0000	1.0000	111.02	5.19	116.21
xception	8	7	1.0000	1.0000	1.0000	1.0000	109.56	5.40	114.96
xception	9	8	1.0000	1.0000	1.0000	1.0000	108.42	6.09	114.51
xception	10	9	1.0000	1.0000	1.0000	1.0000	110.98	5.36	116.34

16.4 Estatísticas Resumo

Tabela 30: Resumo estatístico — Xception

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper
Acuracia	1.0000	0.0000	0.0000	1.0000	1.0000
Precision	1.0000	0.0000	0.0000	1.0000	1.0000
Recall	1.0000	0.0000	0.0000	1.0000	1.0000
F1_score	1.0000	0.0000	0.0000	1.0000	1.0000
Tempo_Treino_s	110.7196	1.8956	0.0171	109.3637	112.0755
Tempo_Pred_s	5.6374	0.4644	0.0824	5.3052	5.9697
Tempo_Total_s	116.3571	2.0847	0.0179	114.8658	117.8483

16.5 Gráficos

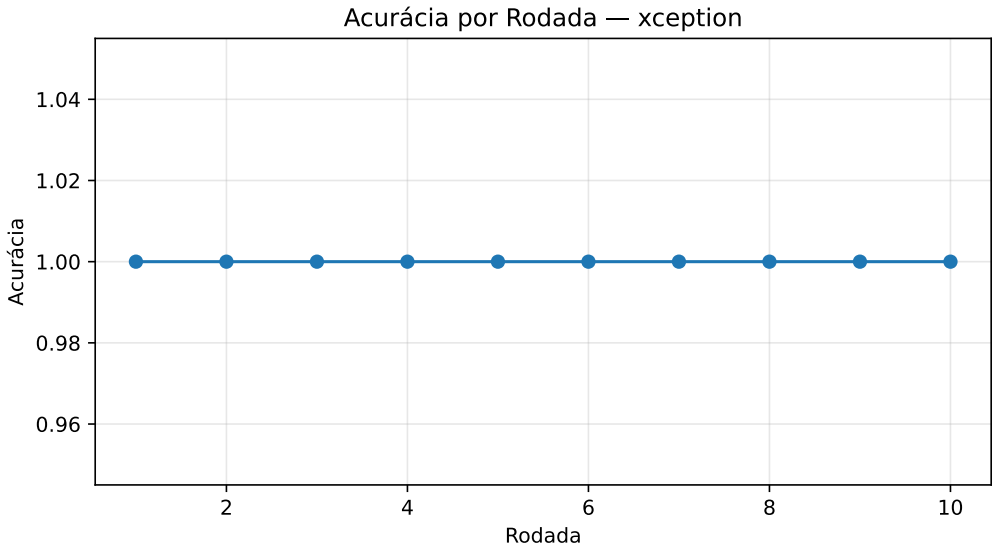

Figura 37: Acurácia por rodada — Xception

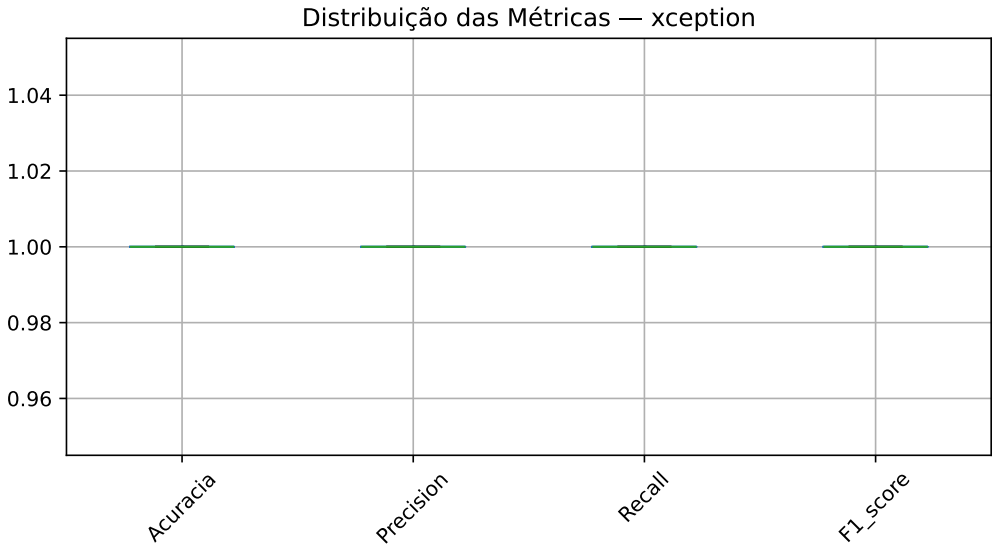

Figura 38: Boxplot das métricas — Xception

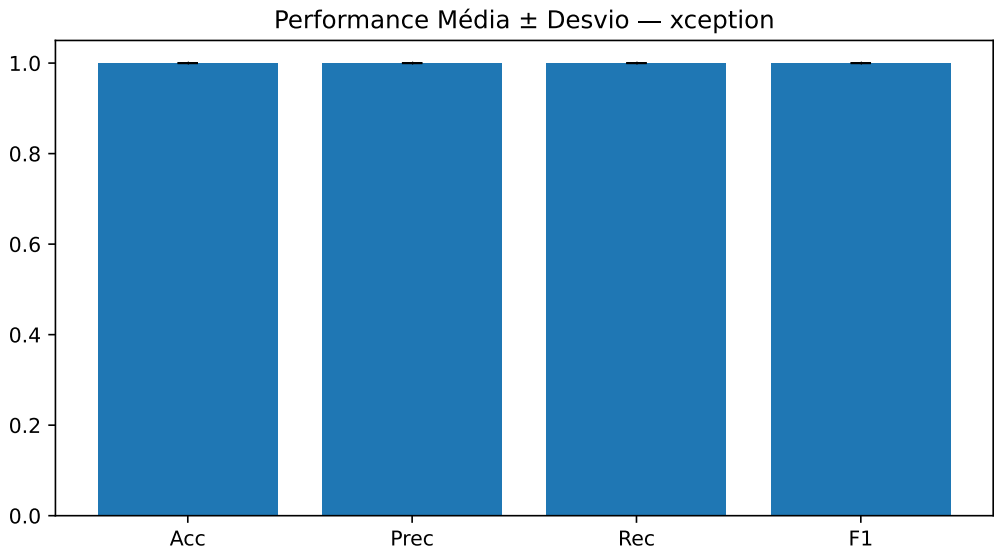

Figura 39: Média ± desvio padrão — Xception

16.6 Análise

A Xception é o segundo modelo com **desempenho perfeito** em todas as rodadas: acurácia, precisão, recall e F1-score iguais a 1,0000 com desvio 0,0000. O custo é o maior de todos os modelos: 110,7 s de treinamento e 116,4 s no total. Para esta tarefa binária, a Xception alcança o limite superior do desempenho, mas com o maior tempo de processamento.

17 Análise Comparativa dos 13 Modelos

17.1 Tabela Comparativa de Médias — Métricas de Classificação

Tabela 31: Média das métricas de classificação por modelo (10 rodadas)

Metrica Modelo	Acuracia	F1_score	Precision	Recall
cnn_from_scratch	0.7400	0.7267	0.7882	0.7400
densenet121	0.9833	0.9833	0.9842	0.9833
efficientnetb0	0.9983	0.9983	0.9984	0.9983
efficientnetb1	0.9983	0.9983	0.9984	0.9983
efficientnetb2	1.0000	1.0000	1.0000	1.0000
inceptionv3	0.9833	0.9833	0.9841	0.9833
mobilenetv2	0.9700	0.9699	0.9723	0.9700
mobilenetv3large	0.9700	0.9699	0.9717	0.9700
mobilenetv3small	0.9650	0.9641	0.9721	0.9650
nasnetmobile	0.9683	0.9683	0.9700	0.9683
resnet50	0.9600	0.9597	0.9650	0.9600
xception	1.0000	1.0000	1.0000	1.0000

17.2 Tabela Comparativa de Médias — Tempos de Execução

Tabela 32: Média dos tempos de execução por modelo, em segundos (10 rodadas)

Metrica Modelo	Tempo_Pred_s	Tempo_Total_s	Tempo_Treino_s
cnn_from_scratch	6.5507	40.9079	34.3572
densenet121	11.3072	68.5330	57.2258
efficientnetb0	7.8116	54.9136	47.1019
efficientnetb1	9.7962	64.0731	54.2768
efficientnetb2	9.9932	65.7119	55.7187
inceptionv3	8.0358	107.2421	99.2064
mobilenetv2	5.8863	46.1598	40.2735
mobilenetv3large	6.5665	49.2361	42.6697
mobilenetv3small	5.8935	46.0466	40.1531
nasnetmobile	15.2543	82.3962	67.1419
resnet50	7.9943	56.3756	48.3813
xception	5.6374	116.3571	110.7196

17.3 Tabela Resumo Consolidada dos 13 Modelos

A tabela a seguir apresenta todos os resumos estatísticos de todos os 13 modelos concatenados, incluindo média, desvio padrão, coeficiente de variação e intervalo de confiança de 95% para cada métrica.

Tabela 33: Resumo estatístico consolidado dos 13 modelos

Metrica	Media	Desvio_Padrao	Coef_Variacao	IC95_Lower	IC95_Upper	Modelo
Acuracia	0.7400	0.0903	0.1221	0.6754	0.8046	cnn_from_scratch
Precision	0.7882	0.0846	0.1074	0.7277	0.8488	cnn_from_scratch
Recall	0.7400	0.0903	0.1221	0.6754	0.8046	cnn_from_scratch
F1_score	0.7267	0.0986	0.1356	0.6562	0.7972	cnn_from_scratch
Tempo_Treino_s	34.3572	15.0289	0.4374	23.6069	45.1075	cnn_from_scratch
Tempo_Pred_s	6.5507	9.3739	1.4310	-0.1545	13.2559	cnn_from_scratch
Tempo_Total_s	40.9079	23.6063	0.5771	24.0222	57.7937	cnn_from_scratch
Acuracia	0.9700	0.0205	0.0211	0.9553	0.9847	mobilenetv2
Precision	0.9723	0.0180	0.0185	0.9594	0.9852	mobilenetv2
Recall	0.9700	0.0205	0.0211	0.9553	0.9847	mobilenetv2
F1_score	0.9699	0.0206	0.0212	0.9552	0.9847	mobilenetv2
Tempo_Treino_s	40.2735	3.0055	0.0746	38.1236	42.4234	mobilenetv2
Tempo_Pred_s	5.8863	0.4626	0.0786	5.5554	6.2172	mobilenetv2
Tempo_Total_s	46.1598	2.9721	0.0644	44.0338	48.2857	mobilenetv2
Acuracia	0.9983	0.0053	0.0053	0.9946	1.0021	efficientnetb0
Precision	0.9984	0.0051	0.0051	0.9947	1.0020	efficientnetb0
Recall	0.9983	0.0053	0.0053	0.9946	1.0021	efficientnetb0
F1_score	0.9983	0.0053	0.0053	0.9946	1.0021	efficientnetb0
Tempo_Treino_s	47.1019	1.0835	0.0230	46.3269	47.8770	efficientnetb0
Tempo_Pred_s	7.8116	0.4903	0.0628	7.4609	8.1623	efficientnetb0
Tempo_Total_s	54.9136	0.9837	0.0179	54.2099	55.6172	efficientnetb0
Acuracia	0.9983	0.0053	0.0053	0.9946	1.0021	efficientnetb1
Precision	0.9984	0.0051	0.0051	0.9947	1.0020	efficientnetb1
Recall	0.9983	0.0053	0.0053	0.9946	1.0021	efficientnetb1
F1_score	0.9983	0.0053	0.0053	0.9946	1.0021	efficientnetb1
Tempo_Treino_s	54.2768	0.7713	0.0142	53.7251	54.8285	efficientnetb1
Tempo_Pred_s	9.7962	0.3557	0.0363	9.5418	10.0506	efficientnetb1
Tempo_Total_s	64.0731	0.6945	0.0108	63.5763	64.5698	efficientnetb1
Acuracia	1.0000	0.0000	0.0000	1.0000	1.0000	efficientnetb2
Precision	1.0000	0.0000	0.0000	1.0000	1.0000	efficientnetb2
Recall	1.0000	0.0000	0.0000	1.0000	1.0000	efficientnetb2
F1_score	1.0000	0.0000	0.0000	1.0000	1.0000	efficientnetb2
Tempo_Treino_s	55.7187	0.9358	0.0168	55.0493	56.3881	efficientnetb2
Tempo_Pred_s	9.9932	0.3444	0.0345	9.7469	10.2395	efficientnetb2
Tempo_Total_s	65.7119	0.8992	0.0137	65.0687	66.3551	efficientnetb2
Acuracia	0.9600	0.0387	0.0403	0.9324	0.9876	resnet50
Precision	0.9650	0.0323	0.0335	0.9419	0.9882	resnet50
Recall	0.9600	0.0387	0.0403	0.9324	0.9876	resnet50
F1_score	0.9597	0.0390	0.0407	0.9318	0.9877	resnet50
Tempo_Treino_s	48.3813	2.6453	0.0547	46.4891	50.2736	resnet50
Tempo_Pred_s	7.9943	1.4309	0.1790	6.9707	9.0178	resnet50
Tempo_Total_s	56.3756	2.8893	0.0513	54.3089	58.4423	resnet50
Acuracia	0.9833	0.0136	0.0138	0.9736	0.9931	densenet121
Precision	0.9842	0.0128	0.0130	0.9750	0.9933	densenet121
Recall	0.9833	0.0136	0.0138	0.9736	0.9931	densenet121
F1_score	0.9833	0.0136	0.0139	0.9736	0.9931	densenet121
Tempo_Treino_s	57.2258	1.7831	0.0312	55.9503	58.5012	densenet121
Tempo_Pred_s	11.3072	0.2795	0.0247	11.1073	11.5072	densenet121
Tempo_Total_s	68.5330	1.8383	0.0268	67.2180	69.8480	densenet121
Acuracia	0.9650	0.0631	0.0654	0.9199	1.0101	mobilenetv3small
Precision	0.9721	0.0457	0.0470	0.9394	1.0048	mobilenetv3small
Recall	0.9650	0.0631	0.0654	0.9199	1.0101	mobilenetv3small
F1_score	0.9641	0.0656	0.0680	0.9172	1.0110	mobilenetv3small
Tempo_Treino_s	40.1531	0.6306	0.0157	39.7021	40.6042	mobilenetv3small
Tempo_Pred_s	5.8935	0.4665	0.0792	5.5598	6.2271	mobilenetv3small
Tempo_Total_s	46.0466	0.8977	0.0195	45.4045	46.6887	mobilenetv3small
Acuracia	0.9700	0.0312	0.0322	0.9477	0.9923	mobilenetv3large
Precision	0.9717	0.0285	0.0294	0.9513	0.9921	mobilenetv3large
Recall	0.9700	0.0312	0.0322	0.9477	0.9923	mobilenetv3large
F1_score	0.9699	0.0314	0.0324	0.9475	0.9924	mobilenetv3large
Tempo_Treino_s	42.6697	0.7924	0.0186	42.1029	43.2365	mobilenetv3large
Tempo_Pred_s	6.5665	0.4238	0.0645	6.2633	6.8696	mobilenetv3large
Tempo_Total_s	49.2361	0.7459	0.0151	48.7026	49.7697	mobilenetv3large
Acuracia	0.9683	0.0254	0.0262	0.9502	0.9865	nasnetmobile
Precision	0.9700	0.0237	0.0244	0.9530	0.9869	nasnetmobile
Recall	0.9683	0.0254	0.0262	0.9502	0.9865	nasnetmobile

17.4 Discussão Geral

17.4.1 Modelos de Melhor Desempenho

Os modelos **EfficientNetB2** e **Xception** atingiram acurácia perfeita (1,0000) em todas as 10 rodadas, com desvio padrão zero. Eles representam a fronteira superior de desempenho para esta tarefa binária.

17.4.2 Família EfficientNet

A família EfficientNet demonstrou excelente desempenho em todos os três variantes:

- **B0**: acurácia 0,9983 — melhor custo-benefício da família
- **B1**: acurácia 0,9983 — sem ganho em relação ao B0, maior custo
- **B2**: acurácia 1,0000 — melhor desempenho, custo moderado

17.4.3 Modelos Leves (*Mobile*)

MobileNetV2, MobileNetV3Large e NASNetMobile apresentam bom desempenho (acurácia $\geq$ 0,97) com tempos de treinamento reduzidos ($\approx$ 40–50 s), sendo adequados para cenários com restrições computacionais.

17.4.4 Impacto da Arquitetura CNN do Zero

A comparação entre as CNNs treinadas do zero e os modelos de TL ilustra claramente o **poder do Transfer Learning**: todos os 11 backbones pré-treinados superaram as CNNs do zero, que atingiram no máximo 0,74 de acurácia. Isso confirma que os pesos aprendidos no ImageNet transferem representações visuais úteis mesmo para domínios específicos como moda.

17.4.5 Relação Desempenho $\times$ Tempo

Tabela 34: Classificação dos modelos por desempenho e custo computacional

Modelo	Acurácia Média	Tempo Treino (s)	Classificação
EfficientNetB2	1.0000	55.7	Melhor desempenho
Xception	1.0000	110.7	Melhor desempenho, mais lento
EfficientNetB0	0.9983	47.1	Ótimo custo-benefício
EfficientNetB1	0.9983	54.3	Similar ao B0, mais lento
DenseNet121	0.9833	57.2	Alto desempenho
InceptionV3	0.9833	99.2	Alto desempenho, lento
MobileNetV2	0.9700	40.3	Bom, muito rápido
MobileNetV3L	0.9700	42.7	Bom, rápido
NASNetMobile	0.9683	67.1	Bom, lento na inferência
MobileNetV3S	0.9650	40.2	Aceitável, mais variável
ResNet50	0.9600	48.4	Bom, CV moderado
CNN (lr=5e-4)	0.7400	34.4	Fraco, muito variável
CNN (lr=1e-4)	0.6567	36.9	Fraco, muito variável

18 Conclusões

Este trabalho avaliou sistematicamente 13 modelos de CNNs para a classificação binária de imagens de moda (bolsas *vs.* calças), utilizando o protocolo Repeated Holdout com 10 sementes, garantindo a replicabilidade e robustez estatística dos resultados.

Os principais achados são:

1. **Transfer Learning supera CNNs do zero**: todos os 11 backbones pré-treinados no ImageNet superaram significativamente as CNNs treinadas do zero, confirmando a eficácia do TL mesmo para domínios específicos.
2. **EfficientNetB2 e Xception**: atingiram desempenho perfeito em todas as rodadas. O B2 é preferível em custo-benefício pois é mais rápido.
3. **EfficientNetB0**: melhor relação desempenho/custo entre os modelos avaliados, com acurácia de 0,9983 e treinamento em ≈ 47 s.
4. **MobileNetV2**: ideal para aplicações com restrições de tempo, atingindo 0,9700 de acurácia em apenas ≈ 40 s.
5. **Taxa de aprendizado nas CNNs do zero**: a taxa 5×10^{-4} superou 1×10^{-4}, mas ambas apresentaram alta variabilidade e desempenho muito inferior ao TL.

Recomendação: Para implantação em produção, o **EfficientNetB2** é o modelo recomendado, com acurácia perfeita e tempo de treinamento razoável. Para cenários com restrições de hardware, o **MobileNetV2** ou o **EfficientNetB0** são alternativas eficientes.

18 Referências

18 Referências

[1] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] Y. LeCun, Y. Bengio, G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.

[3] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*. MIT Press, 2016. Disponível em: https://www.deeplearningbook.org

[4] V. Nair, G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," *ICML*, pp. 807–814, 2010.

[5] X. Glorot, A. Bordes, Y. Bengio, "Deep sparse rectifier neural networks," *AISTATS*, vol. 15, pp. 315–323, 2011.

[6] P. McCullagh, "Regression models for ordinal data," *Journal of the Royal Statistical Society: Series B*, vol. 42, no. 2, pp. 109–127, 1980.

[7] J. S. Bridle, "Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition," in *Neurocomputing: Algo-*

rithms, Architectures and Applications, F. Fogelman-Soulié, J. Hérault (Eds.), Springer, pp. 227–236, 1990.

[8] M. D. Zeiler, R. Fergus, "Visualizing and understanding convolutional networks," *ECCV*, pp. 818–833, 2014.

[9] M. Lin, Q. Chen, S. Yan, "Network in network," *ICLR*, 2014. arXiv:1312.4400.

[10] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[11] S. Ioffe, C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *ICML*, pp. 448–456, 2015.

[12] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[13] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[14] R. R. Bouckaert, E. Frank, "Evaluating the replicability of significance tests for comparing learning algorithms," *PAKDD*, pp. 3–12, 2004.

[15] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.

[16] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.

[17] M. Tan, Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *ICML*, 2019.

[18] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[19] G. Huang, Z. Liu, L. van der Maaten, K. Q. Weinberger, "Densely Connected Convolutional Networks," *CVPR*, 2017.

[20] A. Howard, R. Pang, H. Adam, Q. V. Le, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, "Searching for MobileNetV3," *ICCV*, 2019.

[21] B. Zoph, V. Vasudevan, J. Shlens, Q. V. Le, "Learning Transferable Architectures for Scalable Image Recognition," *CVPR*, 2018.

[22] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," *CVPR*, 2016.

[23] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," *CVPR*, 2017.